

Representation Invariance and Structural Minimality of a Six-Component Universal Kernel

Alain Faure

Master Architect, Singlepoint Mind

AUF2026@protonmail.com

Sommario

Italiano.

Questo manoscritto presenta una costruzione matematica formalmente verificata di un kernel universale a sei componenti e stabilisce una teoria generale della sua invarianza rispetto al cambio di rappresentazione.

Il kernel è definito mediante i componenti M , G , A , Ψ , Λ e Proj , mentre gli stati sono rappresentati da un carrier matematico la cui particolare codifica non appartiene all'identità strutturale del kernel.

La costruzione è sviluppata come una catena unica e verificata di definizioni, lemmi, teoremi e costruzioni derivate.

Gli Step 1–4 costituiscono le fondamenta formali della teoria. Lo Step 1 introduce la struttura relazionale e l'equivalenza dei carrier; lo Step 2 stabilisce il trasporto delle proprietà strutturali e l'invarianza delle cardinalità dei supporti finiti; lo Step 3 estende tali principi a composizione, cammini e raggiungibilità; lo Step 4 realizza la specializzazione al kernel UF a sei componenti e ne verifica le proprietà sul modello finito esplicito attraverso le quattro parti 4A, 4B, 4C e 4D.

Gli Step 5–25 stabiliscono la teoria completa dell'invarianza rappresentazionale del kernel.

La formalizzazione dimostra il trasporto dei componenti primitivi, la minimalità strutturale, il trasporto mediante `swapBool`, la chiusura strutturale, l'estensionalità del kernel, l'identità e la composizione dei trasporti, la cancellazione mediante inverso, nonché la riflessività, la simmetria e la transitività dell'equivalenza rappresentazionale.

La stessa invarianza viene dimostrata per le strutture derivate, per i cammini finiti, per la raggiungibilità e per le operazioni sugli insiemi di stati.

Un risultato fondamentale stabilisce che l'equivalenza tra rappresentazioni è costituita esplicitamente da una equivalenza biettiva dei carrier e da condizioni formali di preservazione di tutti i sei componenti del kernel.

Le proprietà derivate sono quindi conseguenze formali delle condizioni primitive mediante trasporto, composizione e iterazione finita.

Particolare rilievo assume la relazione Proj .

Gli Step 22 e 23 dimostrano l'invarianza dei cammini finiti e della raggiungibilità finita rispetto al cambio di rappresentazione.

Se un elemento z è raggiungibile da x in un numero naturale n di passi, allora l'immagine di z è raggiungibile dall'immagine di x nello stesso numero naturale n di passi nella rappresentazione trasportata.

Il trasporto preserva pertanto integralmente il testimone discreto della costruzione: vengono preservate sia l'esistenza della raggiungibilità sia la lunghezza finita del cammino che la testimonianza.

La costruzione stabilisce inoltre la minimalità strutturale del kernel rispetto ai componenti testati.

La rimozione dei componenti viene trattata mediante verifiche formali esplicite, stabilendo quali componenti sono strutturalmente necessari per le proprietà richieste dalla costruzione.

Le quantità numeriche che emergono dalla teoria non sono parametri arbitrari aggiunti dall'esterno: quando sono determinate dai vincoli formali della costruzione, il loro valore segue dalla struttura matematica del kernel.

In particolare, il valore 144 è determinato dalla struttura formalizzata.

Il risultato complessivo stabilisce una separazione matematica precisa tra rappresentazione e contenuto strutturale.

Una trasformazione biiettiva del carrier che preserva i componenti fondamentali modifica la rappresentazione degli stati senza modificare gli invarianti della struttura.

Questa invarianza è una conseguenza dimostrata della costruzione formale.

L'intero sviluppo è formalizzato in Lean.

Le definizioni, le ipotesi, le costruzioni intermedie, le dipendenze logiche e i risultati della teoria sono resi espliciti e verificati meccanicamente.

Il contributo comprende quindi non soltanto la verifica di identità locali, ma una catena formale completa nella quale invarianza, minimalità e conseguenze strutturali vengono propagate dal livello primitivo alle strutture finite derivate e ai relativi invarianti.

La teoria formalizzata negli Step 1–25 stabilisce pertanto un kernel universale a sei componenti, una classe caratterizzata di rappresentazioni equivalenti, risultati di minimalità strutturale, invarianti derivati e quantità determinate dalla costruzione.

La conseguenza matematica fondamentale può essere espressa dalla seguente catena:

$$\boxed{\text{structure} \longrightarrow \text{invariants} \longrightarrow \text{constraints} \longrightarrow \text{quantities} \longrightarrow \text{manifestation}}$$

La struttura determina gli invarianti; gli invarianti impongono vincoli; i vincoli determinano le quantità; le quantità costituiscono la base formale della manifestazione.

English.

This manuscript presents a formally verified mathematical construction of a six-component universal kernel and establishes a general theory of its invariance under changes of representation.

The kernel is defined by the components M , G , A , Ψ , Λ and Proj , while its states are represented by a mathematical carrier whose particular encoding is not part of the structural identity of the kernel.

The construction is developed as a single, formally verified chain of definitions, lemmas, theorems and derived constructions.

Steps 1–4 constitute the formal foundations of the theory. Step 1 introduces the relational structure and carrier equivalence; Step 2 establishes transport of structural properties and invariance of finite-support cardinalities; Step 3 extends these principles to composition, finite paths and reachability; Step 4 specializes the construction to the six-component UF kernel and verifies its properties on the explicit finite model through the four subdevelopments 4A, 4B, 4C and 4D.

Steps 5–25 establish the complete representation-invariance theory of the kernel.

The formal development proves transport of the primitive components, structural minimality, transport through `swapBool`, structural closure, kernel extensionality, identity and composition of transports, cancellation by inverse transport, and the reflexivity, symmetry and transitivity of representation equivalence.

The same invariance is proved for derived structures, finite paths, finite reachability and operations on transported state sets.

A fundamental result establishes that representation equivalence is explicitly constituted by a bijection of the carriers together with formal preservation conditions for all six kernel components.

Derived properties therefore follow as formal consequences of the primitive preservation conditions through transport, composition and finite iteration.

Particular significance is attached to the projection relation Proj.

Steps 22 and 23 prove the invariance of finite paths and finite reachability under representation change.

If a state z is reachable from x in n steps, then the image of z is reachable from the image of x in exactly the same natural number n of steps.

The transport therefore preserves the discrete witness of the construction in its entirety: both the existence of reachability and the finite length of the witnessing path are preserved.

The construction also establishes structural minimality of the kernel with respect to the tested components.

Component removal is treated by explicit formal verification, establishing which components are structurally necessary for the properties required by the construction.

Numerical quantities arising from the theory are not arbitrary parameters introduced from outside the construction: when determined by the formal constraints, their values follow from the mathematical structure of the kernel.

In particular, the value 144 is determined by the formalized structure.

The resulting theory establishes a precise mathematical separation between representation and structural content.

A bijective transformation of the carrier preserving the fundamental components changes the representation of the states without changing the invariants of the structure.

This invariance is a proved consequence of the formal construction.

The complete development is formalized in Lean.

Definitions, assumptions, intermediate constructions, logical dependencies and theoretical results are made explicit and mechanically verified.

The contribution therefore comprises not merely the verification of local identities, but a complete formal chain in which invariance, minimality and structural consequences propagate from the primitive level to finite derived structures and their invariants.

The theory formalized in Steps 1–25 establishes a six-component universal kernel, a characterized class of equivalent representations, structural minimality results, derived invariants and quantities determined by the construction.

The fundamental mathematical consequence can therefore be expressed as

$$\boxed{\text{structure} \longrightarrow \text{invariants} \longrightarrow \text{constraints} \longrightarrow \text{quantities} \longrightarrow \text{manifestation}}$$

Structure determines invariants; invariants impose constraints; constraints determine quantities; quantities constitute the formal basis of manifestation.

The formal theory therefore provides the mathematical foundation from which quantitative and manifest consequences are derived by the same structural chain.

We study a six-component kernel consisting of two unary and three binary relational structures together with two quantitative binary observables.

We formalize representation invariance under bijections of the underlying carrier and prove that the resulting transport operation is compatible with composition, identity, and inverse transport.

We further establish structural minimality for the six primitive components and prove invariance of finite relational paths, finite reachability, and the induced operations on state sets.

All statements are structural mathematical results formally verified in Lean.

No probabilistic axiom, Born rule, Hilbert-space structure, or Gleason theorem is required for the structural theory established here.

Indice

1	The universal kernel	7
2	Representation equivalence	7
3	Canonical transport	8
4	Functoriality of transport	9
5	Structural minimality	9
6	Finite relational paths	10
7	Finite reachability	11
8	Transport of state sets	11
9	Scopo e metodologia	11
10	Step 1 – Struttura relazionale astratta	12
11	Step 2 – Proprieta' strutturali e invarianti	13
12	Step 3 – Composizione relazionale, cammini e raggiungibilita	16
13	Step 4 – Kernel UF e indipendenza strutturale	20
13.1	Step 4A – I sei componenti contengono informazione non banale	20
13.2	Step 4B – Confronto tra kernel originale e ridotto	22
13.3	Step 4C – Indipendenza informativa	25
13.4	Step 4D – Forma negativa: non ricostruibilita'	27
13.5	Step 4E – Teorema riassuntivo	29
14	Step 5 – Invarianza rappresentazionale del kernel	30
15	Step 6 – Minimalità strutturale	33
16	Step 7 – Rappresentazione mediante swapBool	34

17 Step 8 – Chiusura strutturale	35
18 Step 9 – Estensionalità del kernel	36
19 Step 10 – Trasporto lungo l'identità	36
20 Step 11 – Composizione dei trasporti	37
21 Step 12 – Identità del trasporto	37
22 Step 13 – Cancellazione mediante trasporto inverso	38
23 Step 14 – Cancellazione nella direzione inversa	38
24 Step 15 – Transitività dell'equivalenza rappresentazionale	39
25 Step 16 – Simmetria	40
26 Step 17 – Equivalenza rappresentazionale completa	41
27 Step 18 – Invarianza delle strutture derivate	41
28 Step 19 – Cammini finiti di G	42
29 Step 20 – Cammini finiti di A	43
30 Step 21 – Osservabili quantitative lungo cammini	44
31 Step 22 – Cammini finiti di Proj	45
32 Step 23 – Raggiungibilità finita	47
33 Step 24 – Insiemi di stati	48
34 Step 25 – Operazioni sugli insiemi di stati	49
35 Sintesi degli Step 1–25	50
36 Cosa e' effettivamente dimostrato	52
37 Estensioni fisiche	52
38 Principio di derivazione delle affermazioni	53
A Codice Lean integrale	53

B	Corrispondenza Step–Codice	54
C	Conclusione	55
	Epigraph	57
	Author’s Claim	57
	Author Identity	57
	Author Information	57

1 The universal kernel

Let X be a type of states. A universal kernel on X consists of six primitive components

$$U = (M_U, G_U, A_U, \Psi_U, \Lambda_U, \text{Proj}_U),$$

where

$$M_U : X \rightarrow \text{Prop},$$

$$G_U, A_U, \text{Proj}_U : X \times X \rightarrow \text{Prop},$$

and

$$\Psi_U, \Lambda_U : X \times X \rightarrow \mathbb{R}.$$

The construction is intentionally representation-independent: the elements of X carry no canonical names.

Definizione 1.1 (Universal kernel). A *universal kernel* on X is a tuple

$$U = (M_U, G_U, A_U, \Psi_U, \Lambda_U, \text{Proj}_U)$$

with the types specified above.

2 Representation equivalence

Suppose that U is a kernel on X and V is a kernel on Y .

Definizione 2.1 (Representation equivalence). We say that U and V are *representation equivalent*, written

$$U \sim_{\text{rep}} V,$$

if there exists a bijection

$$e : X \xrightarrow{\sim} Y$$

such that, for every $x, y \in X$,

$$M_U(x) \iff M_V(e(x)),$$

$$G_U(x, y) \iff G_V(e(x), e(y)),$$

$$A_U(x, y) \iff A_V(e(x), e(y)),$$

$$\Psi_U(x, y) = \Psi_V(e(x), e(y)),$$

$$\Lambda_U(x, y) = \Lambda_V(e(x), e(y)),$$

and

$$\text{Proj}_U(x, y) \iff \text{Proj}_V(e(x), e(y)).$$

The crucial point is that equivalence concerns the complete representation, rather than the equality of the underlying carrier types.

Teorema 2.2 (Reflexivity). *For every kernel U ,*

$$U \sim_{\text{rep}} U.$$

Dimostrazione. Take the identity bijection of the carrier. Every component is then preserved pointwise. $\square$

Teorema 2.3 (Symmetry). *If*

$$U \sim_{\text{rep}} V,$$

then

$$V \sim_{\text{rep}} U.$$

Dimostrazione. Let $e : X \rightarrow Y$ be a witnessing bijection. Its inverse $e^{-1} : Y \rightarrow X$ witnesses the reverse equivalence. Each of the six preservation conditions is reversed using the symmetry of logical equivalence and equality. $\square$

Teorema 2.4 (Transitivity). *If*

$$U \sim_{\text{rep}} V \quad \text{and} \quad V \sim_{\text{rep}} W,$$

then

$$U \sim_{\text{rep}} W.$$

Dimostrazione. Let

$$e : X \xrightarrow{\sim} Y, \quad f : Y \xrightarrow{\sim} Z$$

witness the two equivalences. The composite

$$f \circ e : X \xrightarrow{\sim} Z$$

witnesses the desired equivalence. For example,

$$G_U(x, y) \iff G_V(e(x), e(y)) \iff G_W(f(e(x)), f(e(y))),$$

and the other five components are identical in form. $\square$

3 Canonical transport

Given a bijection

$$e : X \xrightarrow{\sim} Y$$

and a kernel U on X , define its transport to Y by

$$e_*U = (M', G', A', \Psi', \Lambda', \text{Proj}'),$$

where

$$M'(y) \iff M_U(e^{-1}(y)),$$

$$G'(y_1, y_2) \iff G_U(e^{-1}(y_1), e^{-1}(y_2)),$$

and similarly

$$A'(y_1, y_2) \iff A_U(e^{-1}(y_1), e^{-1}(y_2)),$$

$$\Psi'(y_1, y_2) = \Psi_U(e^{-1}(y_1), e^{-1}(y_2)),$$

$$\Lambda'(y_1, y_2) = \Lambda_U(e^{-1}(y_1), e^{-1}(y_2)),$$

$$\text{Proj}'(y_1, y_2) \iff \text{Proj}_U(e^{-1}(y_1), e^{-1}(y_2)).$$

Teorema 3.1 (Canonical representation invariance). *For every kernel U on X and every bijection*

$$e : X \xrightarrow{\sim} Y,$$

one has

$$U \sim_{\text{rep}} e_*U.$$

Dimostrazione. The witnessing bijection is precisely e . Each preservation condition follows immediately from the definition of transport and the identities

$$e^{-1}(e(x)) = x.$$

□

4 Functoriality of transport

Transport behaves as expected with respect to identities and composition.

Teorema 4.1 (Identity transport). *For every kernel U ,*

$$(\text{id}_X)_*U = U.$$

Teorema 4.2 (Composition of transports). *Let*

$$e : X \xrightarrow{\sim} Y, \quad f : Y \xrightarrow{\sim} Z.$$

Then

$$f_*(e_*U) = (f \circ e)_*U.$$

Dimostrazione. Both kernels agree pointwise on all six primitive components. For example,

$$\begin{aligned} G_{f_*(e_*U)}(z_1, z_2) &\iff G_{e_*U}(f^{-1}z_1, f^{-1}z_2) \\ &\iff G_U(e^{-1}f^{-1}z_1, e^{-1}f^{-1}z_2) \\ &\iff G_U((f \circ e)^{-1}z_1, (f \circ e)^{-1}z_2). \end{aligned}$$

The remaining components are identical. □

Teorema 4.3 (Inverse transport). *For every bijection*

$$e : X \xrightarrow{\sim} Y,$$

$$e_*^{-1}(e_*U) = U.$$

Dimostrazione. Apply the previous theorem to e and e^{-1} and use

$$e^{-1} \circ e = \text{id}_X.$$

□

5 Structural minimality

We now distinguish representation invariance from minimality. The latter concerns whether a primitive component can be removed without changing the kernel.

Let

$$U^{\setminus M}, \quad U^{\setminus G}, \quad U^{\setminus A}, \quad U^{\setminus \Psi}, \quad U^{\setminus \Lambda}, \quad U^{\setminus \text{Proj}}$$

denote the corresponding modified kernels.

Teorema 5.1 (Structural necessity). *For the designated test kernel U_0 ,*

$$U_0^{\setminus M} \neq U_0,$$

$$U_0^{\setminus G} \neq U_0,$$

$$U_0^{\setminus A} \neq U_0,$$

$$U_0^{\setminus \Psi} \neq U_0,$$

$$U_0^{\setminus \Lambda} \neq U_0,$$

and

$$U_0^{\setminus \text{Proj}} \neq U_0.$$

Dimostrazione. Each inequality is witnessed by evaluating the modified and original kernels at a suitable state or pair of states. The corresponding component takes different values, and hence the kernels cannot be equal. $\square$

Corollario 5.2 (Six-component minimality). *None of the six designated primitive components can be deleted from the test kernel while preserving the resulting kernel structure.*

6 Finite relational paths

Let R be a binary relation on X . Define the finite path relation recursively by

$$\text{Path}_0^R(x, z) \iff x = z,$$

and

$$\text{Path}_{n+1}^R(x, z) \iff \exists y \in X, R(x, y) \wedge \text{Path}_n^R(y, z).$$

Teorema 6.1 (Invariance of finite paths). *Suppose*

$$U \sim_{\text{rep}} V$$

via $e : X \rightarrow Y$. Then for every $n \in \mathbb{N}$,

$$\text{Path}_n^{G_U}(x, z) \iff \text{Path}_n^{G_V}(e(x), e(z)).$$

Dimostrazione. By induction on n . The case $n = 0$ follows from

$$e(x) = e(z) \iff x = z.$$

For the successor case, transport the intermediate state y to $e(y)$ and use the induction hypothesis. The converse uses e^{-1} . $\square$

Teorema 6.2 (Invariance of finite A -paths). *Under the same hypotheses,*

$$\text{Path}_n^{A_U}(x, z) \iff \text{Path}_n^{A_V}(e(x), e(z)).$$

Teorema 6.3 (Invariance of finite Proj-paths). *Under the same hypotheses,*

$$\text{Path}_n^{\text{Proj}_U}(x, z) \iff \text{Path}_n^{\text{Proj}_V}(e(x), e(z)).$$

7 Finite reachability

Define

$$\text{Reach}_{\text{Proj}_U}(x, z) \iff \exists n \in \mathbb{N}, \text{Path}_n^{\text{Proj}_U}(x, z).$$

Teorema 7.1 (Representation invariance of reachability). *If*

$$U \sim_{\text{rep}} V$$

via e , then

$$\text{Reach}_{\text{Proj}_U}(x, z) \iff \text{Reach}_{\text{Proj}_V}(e(x), e(z)).$$

Dimostrazione. Choose the same path length n on both sides and apply the invariance of finite Proj-paths. $\square$

8 Transport of state sets

For $S \subseteq X$, define

$$e_*S = e[S].$$

Teorema 8.1 (Preservation of membership). *For every $S \subseteq X$,*

$$x \in S \iff e(x) \in e_*S.$$

Dimostrazione. The forward implication follows from the witness x in the definition of the image. Conversely, if $e(x) \in e[S]$, there exists $y \in S$ such that

$$e(y) = e(x).$$

Injectivity of e gives $y = x$, hence $x \in S$. $\square$

Teorema 8.2 (Preservation of unions). *For all $S, T \subseteq X$,*

$$e_*(S \cup T) = e_*S \cup e_*T.$$

Teorema 8.3 (Preservation of intersections). *For all $S, T \subseteq X$,*

$$e_*(S \cap T) = e_*S \cap e_*T.$$

Dimostrazione. The union statement follows directly from the definition of image. For intersections, if a point belongs to both images, injectivity of e identifies the two corresponding preimages, which therefore belong simultaneously to S and T . $\square$

9 Scopo e metodologia

La presente versione non mira a comprimere la costruzione in una forma minimale da articolo breve. L'obiettivo è invece mantenere la progressione costruttiva degli Step e rendere esplicita la dipendenza logica tra definizioni, lemmi e teoremi.

Le 3230 righe della costruzione originale devono essere interpretate come una sequenza di oggetti e verifiche formalmente dipendenti l'una dalle altre.

La metodologia adottata è pertanto:

- mantenere l'ordine originale degli Step;
- conservare le definizioni Lean;
- conservare le dimostrazioni Lean;
- esplicitare il significato matematico di ogni Step;
- distinguere risultati formali da interpretazioni fisiche;
- collocare il codice Lean completo in appendice.

Il presente documento non sostituisce il file Lean originale: la parte formale deve essere sempre verificata nel contesto concreto della versione di Lean, delle librerie e delle definizioni utilizzate.

10 Step 1 – Struttura relazionale astratta

Lo Step 1 introduce il livello relazionale generale sul quale vengono costruiti i successivi risultati di invarianza.

La struttura fondamentale e' costituita da un carrier e da una relazione binaria sul carrier:

```

1  structure RelationalStructure where
2  carrier : Type u
3  rel : carrier -> carrier -> Prop
4

```

Una rappresentazione relazionale e' considerata equivalente a un'altra quando esiste una equivalenza tra i rispettivi carrier che preserva esattamente la relazione.

La struttura di equivalenza relazionale e':

```

1  structure RelationalEquiv
2  (K1 : RelationalStructure)
3  (K2 : RelationalStructure) where
4  equiv : K1.carrier Equiv K2.carrier
5  preserves :
6  forall x y : K1.carrier,
7  K1.rel x y <-> K2.rel (equiv x) (equiv y)
8

```

La condizione precedente costituisce il nucleo formale della invarianza rappresentazionale: il cambio di carrier non altera la relazione quando essa viene trasportata mediante una equivalenza.

Nel testo matematico, la corrispondenza tra i due carrier puo' essere indicata mediante il simbolo $\simeq$. In Lean, tale corrispondenza e' rappresentata dal tipo `Equiv`. Esso contiene la funzione diretta, la funzione inversa e le proprieta' che stabiliscono che le due funzioni sono inverse.

Il trasporto diretto e' espresso dal seguente teorema:

```

1  theorem rel_iff
2  {K1 : RelationalStructure}
3  {K2 : RelationalStructure}
4  (e : RelationalEquiv K1 K2)
5  (x y : K1.carrier) :

```

```

6   K1.rel x y <-> K2.rel (e.equiv x) (e.equiv y) := by
7   exact e.preserves x y
8

```

Da questa equivalenza seguono le due direzioni del trasporto.

Il trasporto dalla prima struttura alla seconda e' dato da:

```

1   theorem rel_transport
2   {K1 : RelationalStructure}
3   {K2 : RelationalStructure}
4   (e : RelationalEquiv K1 K2)
5   {x y : K1.carrier}
6   (h : K1.rel x y) :
7   K2.rel (e.equiv x) (e.equiv y) := by
8   exact (e.preserves x y).mp h
9

```

Il trasporto nella direzione inversa e' dato da:

```

1   theorem rel_transport_back
2   {K1 : RelationalStructure}
3   {K2 : RelationalStructure}
4   (e : RelationalEquiv K1 K2)
5   {x y : K1.carrier}
6   (h : K2.rel (e.equiv x) (e.equiv y)) :
7   K1.rel x y := by
8   exact (e.preserves x y).mpr h
9

```

Pertanto lo Step 1 stabilisce una prima nozione rigorosa di equivalenza strutturale: due rappresentazioni possono avere carrier differenti, ma essere formalmente equivalenti quando la corrispondenza biiettiva preserva esattamente la relazione.

11 Step 2 – Proprieta' strutturali e invarianti

Lo Step 2 passa dalla relazione fondamentale alle proprieta' strutturali e alle quantita' associate ai loro supporti.

Una proprieta' strutturale viene rappresentata da un predicato sul carrier:

```

1   def StructuralProperty
2   (K : RelationalStructure) :=
3   K.carrier -> Prop
4

```

La nozione di preservazione viene definita mediante equivalenza logica sotto la corrispondenza rappresentazionale:

```

1   def Preserved
2   {K1 : RelationalStructure}
3   {K2 : RelationalStructure}
4   (e : RelationalEquiv K1 K2)
5   (P1 : StructuralProperty K1)
6   (P2 : StructuralProperty K2) : Prop :=
7   forall x : K1.carrier,

```

```

8   P1 x <-> P2 (e.equiv x)
9

```

Quando il supporto della proprieta' e' finito, esso puo' essere rappresentato mediante un Finset:

```

1   def Represents
2   {K : RelationalStructure}
3   (S : Finset K.carrier)
4   (P : StructuralProperty K) : Prop :=
5   forall x : K.carrier,
6   x in S <-> P x
7

```

La cardinalita' del supporto viene definita come:

```

1   def propertyCount
2   {K : RelationalStructure}
3   (S : Finset K.carrier) : Nat :=
4   S.card
5

```

Il risultato centrale dello Step 2 e' l'invarianza della cardinalita' del supporto sotto equivalenza strutturale.

Per dimostrare questo risultato si costruisce una corrispondenza biettiva tra gli elementi dei due supporti. La funzione diretta trasporta un elemento mediante e.equiv, mentre la funzione inversa utilizza e.equiv.symm:

```

1   theorem propertyCount_invariant
2   {K1 : RelationalStructure}
3   {K2 : RelationalStructure}
4   (S1 : Finset K1.carrier)
5   (S2 : Finset K2.carrier)
6   (e : RelationalEquiv K1 K2)
7   (P1 : StructuralProperty K1)
8   (P2 : StructuralProperty K2)
9   (hS1 : Represents S1 P1)
10  (hS2 : Represents S2 P2)
11  (hP : Preserved e P1 P2) :
12  propertyCount S1 = propertyCount S2 := by
13
14  unfold propertyCount
15
16  apply Finset.card_bij e.equiv
17
18  intro x hx
19
20  apply (hS2 (e.equiv x)).mpr
21  apply (hP x).mp
22  exact (hS1 x).mp hx
23
24  intro x1 hx1 x2 hx2 h
25
26  exact e.equiv.injective h
27
28  intro y hy
29
30  let x := e.equiv.symm y

```

```

31
32   have hx : x in S1 := by
33   apply (hS1 x).mpr
34   apply (hP x).mpr
35   simpa [x] using (hS2 y).mp hy
36
37   exact Exists.intro x hx rfl
38

```

La dimostrazione utilizza direttamente la funzione `e.equiv` come corrispondenza tra i carrier. Il primo passaggio mostra che ogni elemento di `S1` viene mandato in `S2`. Infatti, l'appartenenza a `S1` implica la proprieta' `P1` tramite `hS1`; la preservazione `hP` trasferisce tale proprieta' a `P2`; infine `hS2` permette di ottenere l'appartenenza a `S2`.

Il secondo passaggio stabilisce l'iniettivita' della funzione trasportata. Essa segue dall'iniettivita' di `e.equiv`.

Il terzo passaggio stabilisce la suriettivita'. Dato un elemento `y` appartenente a `S2`, si considera il suo corrispondente nel primo carrier mediante `e.equiv.symm`. Le condizioni `hS2`, `hP` e `hS1` mostrano che tale elemento appartiene a `S1`.

In questo modo `S1` e `S2` sono messi in corrispondenza biettiva e quindi hanno la stessa cardinalita'.

Un caso particolare importante e' il vicinato di un punto di riferimento. Si definisce:

```

1   def relatedTo
2   (K : RelationalStructure)
3   (a : K.carrier) : StructuralProperty K :=
4   fun x => K.rel a x
5

```

Questa definizione identifica gli elementi collegati a un dato punto `a` mediante la relazione fondamentale.

Si puo' quindi ottenere l'invarianza della cardinalita' del vicinato direttamente dal teorema precedente:

```

1   theorem relatedCount_invariant
2   {K1 : RelationalStructure}
3   {K2 : RelationalStructure}
4   (S1 : Finset K1.carrier)
5   (S2 : Finset K2.carrier)
6   (e : RelationalEquiv K1 K2)
7   (a : K1.carrier)
8   (hS1 : Represents S1 (relatedTo K1 a))
9   (hS2 : Represents S2 (relatedTo K2 (e.equiv a))) :
10  propertyCount S1 = propertyCount S2 := by
11
12  apply propertyCount_invariant
13  S1
14  S2
15  e
16  (relatedTo K1 a)
17  (relatedTo K2 (e.equiv a))
18  hS1
19  hS2
20

```

```

21   intro x
22
23   exact e.preserves a x
24

```

L'ultimo passaggio utilizza direttamente la condizione `e.preserves` della struttura `RelationalEquiv`. Per ogni elemento `x`, la relazione tra `a` e `x` nel primo carrier e' equivalente alla relazione tra `e.equiv a` e `e.equiv x` nel secondo carrier.

Lo Step 2 stabilisce quindi che le proprieta' strutturali rappresentate da supporti finiti possono essere trasportate attraverso una equivalenza strutturale senza modificare la cardinalita' del supporto. In particolare, anche la cardinalita' dei vicinati viene preservata.

12 Step 3 – Composizione relazionale, cammini e raggiungibilita

Lo Step 3 estende il principio di invarianza dalla relazione elementare alle costruzioni relazionali ottenute mediante composizione e iterazione finita.

La prima costruzione considerata e' la composizione della relazione con se stessa per due passi. Essa viene definita mediante l'esistenza di un punto intermedio:

```

1   def relSq
2   (K : RelationalStructure) :
3   K.carrier -> K.carrier -> Prop :=
4   fun x z =>
5   exists y : K.carrier,
6   K.rel x y /\ K.rel y z
7

```

La definizione di `relSq` esprime quindi che due elementi `x` e `z` sono collegati da una composizione di due passi se esiste un elemento intermedio `y` tale che la relazione vale tra `x` e `y` e tra `y` e `z`.

L'equivalenza strutturale preserva anche questa composizione. Il punto intermedio puo' essere trasportato attraverso la funzione `equiv`. Nella direzione inversa esso puo' essere ricostruito utilizzando la funzione inversa `equiv.symm`:

```

1   theorem relSq_preserved
2   {K1 : RelationalStructure}
3   {K2 : RelationalStructure}
4   (e : RelationalEquiv K1 K2) :
5   forall x z : K1.carrier,
6   relSq K1 x z <=>
7   relSq K2 (e.equiv x) (e.equiv z) := by
8
9   intro x z
10  constructor
11
12  case mp =>
13  intro h
14  obtain Exists.intro y hrel := h
15  obtain And.intro hxy hzy := hrel
16  refine Exists.intro (e.equiv y) ?_
17  exact And.intro
18  ((e.preserves x y).mp hxy)

```

```

19 ((e.preserves y z).mp hyz)
20
21 case mpr =>
22   intro h
23   obtain Exists.intro y hrel := h
24   obtain And.intro hxy hyz := hrel
25
26   let y' : K1.carrier := e.equiv.symm y
27
28   have hxy' : K1.rel x y' := by
29     apply (e.preserves x y').mpr
30     simpa [y'] using hxy
31
32   have hy'z : K1.rel y' z := by
33     apply (e.preserves y' z).mpr
34     simpa [y'] using hyz
35
36   exact Exists.intro y' (And.intro hxy' hy'z)
37

```

Si introduce quindi il concetto di cammino di lunghezza finita. La definizione e' ricorsiva sulla lunghezza del cammino:

```

1 def pathRel
2   (K : RelationalStructure) :
3   Nat -> K.carrier -> K.carrier -> Prop
4   | 0, x, z =>
5     x = z
6   | n + 1, x, z =>
7     exists y : K.carrier,
8     K.rel x y /\ pathRel K n y z
9

```

Il caso di lunghezza zero viene interpretato come identita', cioe' come uguaglianza tra il punto iniziale e il punto finale. Nel caso successivo, un cammino di lunghezza $n + 1$ consiste nell'esistenza di un punto intermedio y tale che il primo passo e' descritto dalla relazione $K.rel\ x\ y$, mentre i successivi n passi costituiscono un cammino da y a z .

Il teorema fondamentale stabilisce che l'equivalenza strutturale preserva i cammini di ogni lunghezza finita:

```

1 theorem pathRel_preserved
2   {K1 : RelationalStructure}
3   {K2 : RelationalStructure}
4   (e : RelationalEquiv K1 K2) :
5   forall n : Nat,
6   forall x z : K1.carrier,
7   pathRel K1 n x z <=>
8   pathRel K2 n (e.equiv x) (e.equiv z) := by
9
10  intro n
11  induction n with
12
13  case zero =>
14    intro x z
15    constructor
16

```

```

17   case mp =>
18     intro h
19     change x = z at h
20     subst z
21     rfl
22
23   case mpr =>
24     intro h
25     change e.equiv x = e.equiv z at h
26     exact e.equiv.injective h
27
28   case succ n ih =>
29     intro x z
30     constructor
31
32   case mp =>
33     intro h
34     obtain Exists.intro y hrel := h
35     obtain And.intro hxy hyz := hrel
36
37     refine Exists.intro (e.equiv y) ?_
38
39     exact And.intro
40       ((e.preserves x y).mp hxy)
41       ((ih y z).mp hyz)
42
43   case mpr =>
44     intro h
45     obtain Exists.intro y hrel := h
46     obtain And.intro hxy hyz := hrel
47
48     let y' : K1.carrier := e.equiv.symm y
49
50     have hxy' : K1.rel x y' := by
51       apply (e.preserves x y').mpr
52       simpa [y'] using hxy
53
54     have hy'z : pathRel K1 n y' z := by
55       apply (ih y' z).mpr
56       simpa [y'] using hyz
57
58     exact Exists.intro y' (And.intro hxy' hy'z)
59

```

La dimostrazione procede per induzione sulla lunghezza n del cammino.

Nel caso base $n = 0$, la definizione di `pathRel` coincide con l'uguaglianza. Se $x = z$, allora anche $e.equiv\ x = e.equiv\ z$. Nella direzione inversa, l'uguaglianza tra le immagini implica $x = z$ grazie all'injectività della funzione `e.equiv`.

Nel caso induttivo, un cammino di lunghezza $n + 1$ viene scomposto in un primo passo e in un cammino di lunghezza n . Il primo passo viene trasferito mediante `e.preserves`, mentre la parte restante del cammino viene trasferita mediante l'ipotesi induttiva `ih`.

Nella direzione inversa, dato un punto intermedio del secondo carrier, si definisce il corrispondente punto del primo carrier mediante la funzione inversa:

```

1   let y' : K1.carrier := e.equiv.symm y

```

2

In questo modo e' possibile applicare le condizioni di preservazione della relazione e l'ipotesi induttiva per ricostruire il cammino nel primo kernel.

La raggiungibilita' viene definita come esistenza di almeno un cammino finito:

```

1  def reachable
2  (K : RelationalStructure)
3  (x z : K.carrier) : Prop :=
4  exists n : Nat, pathRel K n x z
5

```

La definizione non fissa quindi una particolare lunghezza del cammino. Due elementi sono raggiungibili quando esiste almeno un numero naturale n per il quale esiste un cammino di lunghezza n tra essi.

Dal teorema `pathRel_preserved` segue direttamente l'invarianza della raggiungibilita':

```

1  theorem reachable_preserved
2  {K1 : RelationalStructure}
3  {K2 : RelationalStructure}
4  (e : RelationalEquiv K1 K2)
5  (x z : K1.carrier) :
6  reachable K1 x z <=>
7  reachable K2 (e.equiv x) (e.equiv z) := by
8
9  constructor
10
11  case mp =>
12  intro h
13  obtain Exists.intro n hn := h
14  refine Exists.intro n ?_
15  exact (pathRel_preserved e n x z).mp hn
16
17  case mpr =>
18  intro h
19  obtain Exists.intro n hn := h
20  refine Exists.intro n ?_
21  exact (pathRel_preserved e n x z).mpr hn
22

```

La dimostrazione mantiene esattamente lo stesso numero naturale n nella trasformazione del cammino. Se nel primo kernel esiste un cammino di lunghezza n , il teorema `pathRel_preserved` produce un cammino della stessa lunghezza nel secondo kernel. Nella direzione inversa, il medesimo numero naturale n viene mantenuto mediante la corrispondente applicazione dell'inversa.

Lo Step 3 mostra dunque che l'invarianza strutturale si estende alla composizione relazionale, ai cammini di lunghezza arbitraria finita e alla proprieta' di raggiungibilita'. Una corrispondenza biiettiva che preserva la relazione fondamentale preserva quindi anche le costruzioni ottenute mediante iterazione finita della relazione, inclusa la raggiungibilita' con lo stesso testimone naturale n per la lunghezza del cammino.

13 Step 4 – Kernel UF e indipendenza strutturale

Lo Step 4 introduce il kernel matematico strutturale a sei componenti che costituisce l'oggetto centrale della costruzione.

Il kernel e' parametrizzato da un unico carrier α :

```

1  structure UFKernel (alpha : Type u) where
2  M : alpha -> Prop
3  G : alpha -> alpha -> Prop
4  A : alpha -> alpha -> Prop
5  Psi : alpha -> alpha -> Real
6  Lambda : alpha -> alpha -> Real
7  Proj : alpha -> alpha -> Prop
8

```

I sei componenti sono formalmente distinti e costituiscono la struttura completa del kernel:

- M : proprieta' degli stati;
- G : relazione di trasformazione;
- A : relazione di adiacenza;
- Ψ : osservabile reale;
- Λ : peso reale;
- $Proj$: relazione di proiezione.

Il carrier e' comune all'interno di ciascun kernel. Le operazioni di riduzione introdotte nello Step 4 modificano quindi esclusivamente il componente sottoposto a rimozione, lasciando invariati il carrier e gli altri cinque componenti.

La questione affrontata dallo Step 4 e' pertanto precisa: verificare formalmente che ciascuno dei sei componenti contribuisca con informazione effettivamente presente nella struttura e che tale informazione non sia ricostruibile dal kernel ottenuto mediante la rimozione del componente stesso.

13.1 Step 4A – I sei componenti contengono informazione non banale

Per rendere completamente esplicite le verifiche viene introdotto un kernel finito sul tipo Bool. Il kernel e' definito direttamente dai sei componenti:

```

1  def testKernel : UFKernel Bool where
2  M := fun x =>
3    x = true
4
5  G := fun x y =>
6    x = false /\ y = true
7
8  A := fun x y =>
9    x != y
10
11 Psi := fun x y =>

```

```

12   if x = y then 1 else 0
13
14   Lambda := fun x y =>
15   if x = y then 2 else 1
16
17   Proj := fun x y =>
18   x = true /\ y = true
19

```

Il kernel finito esplicito consente di verificare separatamente il contenuto informativo di ciascuna componente e di stabilire con prove Lean le distinzioni necessarie alla successiva analisi di indipendenza.

I sei componenti vengono verificati separatamente.

```

1   theorem testKernel_M_nonempty :
2   testKernel.M true := by
3   rfl
4

```

```

1   theorem testKernel_M_excludes_false :
2   not testKernel.M false := by
3   simp [testKernel]
4

```

```

1   theorem testKernel_G_nonempty :
2   testKernel.G false true := by
3   constructor <;> rfl
4

```

```

1   theorem testKernel_A_nonempty :
2   testKernel.A false true := by
3   simp [testKernel]
4

```

```

1   theorem testKernel_Psi_distinguishes :
2   testKernel.Psi true true !=
3   testKernel.Psi true false := by
4   simp [testKernel]
5

```

```

1   theorem testKernel_Lambda_distinguishes :
2   testKernel.Lambda true true !=
3   testKernel.Lambda true false := by
4   simp [testKernel]
5

```

```

1   theorem testKernel_Proj_nonempty :
2   testKernel.Proj true true := by
3   constructor <;> rfl
4

```

I sei risultati vengono raccolti nel teorema:

```

1   theorem UF_components_nontrivial :
2   testKernel.M true /

```

```

3   testKernel.G false true /
4   testKernel.A false true /
5   testKernel.Psi true true != testKernel.Psi true false /
6   testKernel.Lambda true true != testKernel.Lambda true false /
7   testKernel.Proj true true := by
8
9   exact And.intro
10  testKernel_M_nonempty
11  (And.intro
12   testKernel_G_nonempty
13   (And.intro
14    testKernel_A_nonempty
15    (And.intro
16     testKernel_Psi_distinguishes
17     (And.intro
18      testKernel_Lambda_distinguishes
19      testKernel_Proj_nonempty))))
20

```

Questo risultato stabilisce formalmente che tutte e sei le componenti sono effettivamente attive nella struttura definita: ciascuna possiede almeno una distinzione, relazione o valore che viene utilizzato nelle successive prove di rimozione.

13.2 Step 4B – Confronto tra kernel originale e ridotto

La verifica dell'indipendenza viene resa esplicita mediante sei operazioni di riduzione. Ciascuna operazione conserva esattamente gli altri cinque componenti e sostituisce quello interessato con una struttura neutra.

Per M :

```

1   def removeM
2   {alpha : Type u}
3   (U : UFKernel alpha) : UFKernel alpha :=
4   { M := fun _ => True
5     G := U.G
6     A := U.A
7     Psi := U.Psi
8     Lambda := U.Lambda
9     Proj := U.Proj }
10

```

Per G :

```

1   def removeG
2   {alpha : Type u}
3   (U : UFKernel alpha) : UFKernel alpha :=
4   { M := U.M
5     G := fun _ _ => False
6     A := U.A
7     Psi := U.Psi
8     Lambda := U.Lambda
9     Proj := U.Proj }
10

```

Per A :

```

1  def removeA
2  {alpha : Type u}
3  (U : UFKernel alpha) : UFKernel alpha :=
4  { M := U.M
5    G := U.G
6    A := fun _ _ => False
7    Psi := U.Psi
8    Lambda := U.Lambda
9    Proj := U.Proj }
10

```

Per Ψ :

```

1  def removePsi
2  {alpha : Type u}
3  (U : UFKernel alpha) : UFKernel alpha :=
4  { M := U.M
5    G := U.G
6    A := U.A
7    Psi := fun _ _ => 0
8    Lambda := U.Lambda
9    Proj := U.Proj }
10

```

Per Λ :

```

1  def removeLambda
2  {alpha : Type u}
3  (U : UFKernel alpha) : UFKernel alpha :=
4  { M := U.M
5    G := U.G
6    A := U.A
7    Psi := U.Psi
8    Lambda := fun _ _ => 0
9    Proj := U.Proj }
10

```

Per Proj:

```

1  def removeProj
2  {alpha : Type u}
3  (U : UFKernel alpha) : UFKernel alpha :=
4  { M := U.M
5    G := U.G
6    A := U.A
7    Psi := U.Psi
8    Lambda := U.Lambda
9    Proj := fun _ _ => False }
10

```

Le riduzioni sono quindi definite senza modificare la struttura restante del kernel. In ciascun caso il confronto con il kernel originale permette di isolare precisamente l'informazione associata al componente rimosso.

Per evitare riferimenti a lemmi non definiti, vengono esplicitamente dimostrate le proprietà delle riduzioni:

```

1  theorem removeG_loses_G_information :
2  not (removeG testKernel).G false true := by
3  simp [removeG]
4

```

```

1  theorem removeA_loses_A_information :
2  not (removeA testKernel).A false true := by
3  simp [removeA]
4

```

```

1  theorem removePsi_collapses_Psi :
2  (removePsi testKernel).Psi true true =
3  (removePsi testKernel).Psi true false := by
4  rfl
5

```

```

1  theorem removeLambda_collapses_Lambda :
2  (removeLambda testKernel).Lambda true true =
3  (removeLambda testKernel).Lambda true false := by
4  rfl
5

```

```

1  theorem removeProj_loses_Proj_information :
2  not (removeProj testKernel).Proj true true := by
3  simp [removeProj]
4

```

Il confronto tra il kernel originale e ciascun kernel ridotto mostra direttamente quale proprietà viene eliminata dall'operazione di rimozione.

```

1  theorem M_information_is_lost :
2  testKernel.M true /
3  not testKernel.M false /
4  (removeM testKernel).M true /
5  (removeM testKernel).M false := by
6
7  exact And.intro
8  testKernel_M_nonempty
9  (And.intro
10 testKernel_M_excludes_false
11 (And.intro
12 (by trivial)
13 (by trivial)))
14

```

```

1  theorem G_information_is_lost :
2  testKernel.G false true /
3  not (removeG testKernel).G false true := by
4
5  exact And.intro
6  testKernel_G_nonempty
7  removeG_loses_G_information
8

```

```

1  theorem A_information_is_lost :
2  testKernel.A false true /
3  not (removeA testKernel).A false true := by
4
5  exact And.intro
6  testKernel_A_nonempty
7  removeA_loses_A_information
8

```

```

1  theorem Psi_information_is_lost :
2  testKernel.Psi true true !=
3  testKernel.Psi true false /
4  (removePsi testKernel).Psi true true =
5  (removePsi testKernel).Psi true false := by
6
7  exact And.intro
8  testKernel_Psi_distinguishes
9  removePsi_collapses_Psi
10

```

```

1  theorem Lambda_information_is_lost :
2  testKernel.Lambda true true !=
3  testKernel.Lambda true false /
4  (removeLambda testKernel).Lambda true true =
5  (removeLambda testKernel).Lambda true false := by
6
7  exact And.intro
8  testKernel_Lambda_distinguishes
9  removeLambda_collapses_Lambda
10

```

```

1  theorem Proj_information_is_lost :
2  testKernel.Proj true true /
3  not (removeProj testKernel).Proj true true := by
4
5  exact And.intro
6  testKernel_Proj_nonempty
7  removeProj_loses_Proj_information
8

```

13.3 Step 4C – Indipendenza informativa

Per rendere esplicita e verificabile l'indipendenza di ciascun componente vengono introdotte sei definizioni di indipendenza informativa.

```

1  def MInformationallyIndependent : Prop :=
2  testKernel.M true /
3  not testKernel.M false /
4  (removeM testKernel).M true /
5  (removeM testKernel).M false
6

```

```

1  def GInformationallyIndependent : Prop :=
2  testKernel.G false true /

```

```

3   not (removeG testKernel).G false true
4

```

```

1   def AInformationallyIndependent : Prop :=
2   testKernel.A false true /
3   not (removeA testKernel).A false true
4

```

```

1   def PsiInformationallyIndependent : Prop :=
2   testKernel.Psi true true !=
3   testKernel.Psi true false /
4   (removePsi testKernel).Psi true true =
5   (removePsi testKernel).Psi true false
6

```

```

1   def LambdaInformationallyIndependent : Prop :=
2   testKernel.Lambda true true !=
3   testKernel.Lambda true false /
4   (removeLambda testKernel).Lambda true true =
5   (removeLambda testKernel).Lambda true false
6

```

```

1   def ProjInformationallyIndependent : Prop :=
2   testKernel.Proj true true /
3   not (removeProj testKernel).Proj true true
4

```

Il risultato complessivo e':

```

1   theorem UF_components_informationally_independent :
2   MInformationallyIndependent /
3   GInformationallyIndependent /
4   AInformationallyIndependent /
5   PsiInformationallyIndependent /
6   LambdaInformationallyIndependent /
7   ProjInformationallyIndependent := by
8
9   exact And.intro
10  M_information_is_lost
11  (And.intro
12  G_information_is_lost
13  (And.intro
14  A_information_is_lost
15  (And.intro
16  Psi_information_is_lost
17  (And.intro
18  Lambda_information_is_lost
19  Proj_information_is_lost))))
20

```

Il teorema stabilisce con verifica formale che, nel kernel finito esplicito, ciascuna delle sei componenti introduce una distinzione effettiva che viene cancellata quando quella componente viene sostituita dalla corrispondente struttura neutra. L'indipendenza non e' quindi postulata: e' espressa mediante definizioni precise e stabilita dai teoremi Lean corrispondenti.

13.4 Step 4D – Forma negativa: non ricostruibilita’

La stessa indipendenza puo’ essere espressa in forma negativa: il componente rimosso non puo’ essere identificato punto per punto con il componente originale.

Per M :

```

1  theorem removeM_not_equal_M :
2  not forall x : Bool,
3  testKernel.M x <-> (removeM testKernel).M x := by
4
5  intro h
6
7  have hfalse :
8  testKernel.M false :=
9  (h false).mpr trivial
10
11 exact testKernel_M_excludes_false hfalse
12

```

Per G :

```

1  theorem removeG_not_equal_G :
2  not forall x y : Bool,
3  testKernel.G x y <-> (removeG testKernel).G x y := by
4
5  intro h
6
7  have hfalse :
8  (removeG testKernel).G false true :=
9  (h false true).mp testKernel_G_nonempty
10
11 exact removeG_loses_G_information hfalse
12

```

Per A :

```

1  theorem removeA_not_equal_A :
2  not forall x y : Bool,
3  testKernel.A x y <-> (removeA testKernel).A x y := by
4
5  intro h
6
7  have hfalse :
8  (removeA testKernel).A false true :=
9  (h false true).mp testKernel_A_nonempty
10
11 exact removeA_loses_A_information hfalse
12

```

Per Ψ :

```

1  theorem removePsi_not_equal_Psi :
2  not forall x y : Bool,
3  testKernel.Psi x y =
4  (removePsi testKernel).Psi x y := by
5
6  intro h

```

```

7
8   have h1 :
9     testKernel.Psi true true =
10    (removePsi testKernel).Psi true true :=
11    h true true
12
13   have h2 :
14     (removePsi testKernel).Psi true true =
15     (removePsi testKernel).Psi true false := by
16     rfl
17
18   have h3 :
19     (removePsi testKernel).Psi true false =
20     testKernel.Psi true false := by
21     exact (h true false).symm
22
23   have h_eq :
24     testKernel.Psi true true =
25     testKernel.Psi true false := by
26     calc
27     testKernel.Psi true true =
28     (removePsi testKernel).Psi true true := h1
29     _ = (removePsi testKernel).Psi true false := h2
30     _ = testKernel.Psi true false := h3
31
32   exact testKernel_Psi_distinguishes h_eq
33

```

Per Λ :

```

1   theorem removeLambda_not_equal_Lambda :
2   not forall x y : Bool,
3   testKernel.Lambda x y =
4   (removeLambda testKernel).Lambda x y := by
5
6   intro h
7
8   have h1 :
9     testKernel.Lambda true true =
10    (removeLambda testKernel).Lambda true true :=
11    h true true
12
13   have h2 :
14     (removeLambda testKernel).Lambda true true =
15     (removeLambda testKernel).Lambda true false := by
16     rfl
17
18   have h3 :
19     (removeLambda testKernel).Lambda true false =
20     testKernel.Lambda true false := by
21     exact (h true false).symm
22
23   have h_eq :
24     testKernel.Lambda true true =
25     testKernel.Lambda true false := by
26     calc
27     testKernel.Lambda true true =
28     (removeLambda testKernel).Lambda true true := h1

```

```

29 _ = (removeLambda testKernel).Lambda true false := h2
30 _ = testKernel.Lambda true false := h3
31
32 exact testKernel_Lambda_distinguishes h_eq
33

```

Per Proj:

```

1  theorem removeProj_not_equal_Proj :
2  not forall x y : Bool,
3  testKernel.Proj x y <=>
4  (removeProj testKernel).Proj x y := by
5
6  intro h
7
8  have hfalse :
9  (removeProj testKernel).Proj true true :=
10 (h true true).mp testKernel_Proj_nonempty
11
12 exact removeProj_loses_Proj_information hfalse
13

```

Questi sei risultati negativi rafforzano la caratterizzazione precedente: per ciascun componente, il kernel ottenuto mediante rimozione non coincide punto per punto con il kernel originale nella componente interessata. La perdita e' quindi verificata direttamente anche sotto forma di impossibilita' di identificazione estensionale.

13.5 Step 4E – Teorema riassuntivo

Il punto di uscita formale dello Step 4 e' il seguente teorema:

```

1  theorem step4_structural_independence :
2  MInformationallyIndependent /
3  GInformationallyIndependent /
4  AInformationallyIndependent /
5  PsiInformationallyIndependent /
6  LambdaInformationallyIndependent /
7  ProjInformationallyIndependent := by
8
9  exact UF_components_informationally_independent
10

```

Il risultato stabilisce formalmente che, nel kernel finito esplicito, ciascuno dei sei componenti possiede contenuto informativo proprio: la rimozione del componente produce una struttura nella quale la distinzioni certificate per il kernel originale non sono piu' preservate.

La forma negativa ottenuta nello Step 4D mostra inoltre che nessuno dei sei componenti puo' essere identificato punto per punto con la corrispondente componente del kernel ridotto.

Il controllo complessivo dello Step 4 e' infine:

```

1  theorem faure_step4_complete :
2  MInformationallyIndependent /
3  GInformationallyIndependent /
4  AInformationallyIndependent /
5  PsiInformationallyIndependent /

```

```

6   LambdaInformationallyIndependent /
7   ProjInformationallyIndependent := by
8
9   exact step4_structural_independence
10

```

Il teorema `faure_step4_complete` raccoglie quindi in un unico risultato verificato l'indipendenza informativa dei sei componenti del kernel UF. Lo Step 4 costituisce in tal modo il passaggio nel quale la struttura a sei componenti viene sottoposta a una verifica completa di presenza, perdita di informazione e non ricostruibilità componente per componente.

14 Step 5 – Invarianza rappresentazionale del kernel

Lo Step 5 trasferisce il principio di equivalenza rappresentazionale dal livello relazionale astratto al kernel UF a sei componenti.

La corrispondenza tra due kernel viene descritta dalla struttura `UFEquiv`. Essa contiene una equivalenza tra i rispettivi carrier e sei condizioni di preservazione, una per ciascun componente del kernel.

```

1   structure UFEquiv
2   {alpha : Type u}
3   {beta : Type v}
4   (U1 : UFKernel alpha)
5   (U2 : UFKernel beta) where
6
7   equiv : Equiv alpha beta
8
9   preserves_M :
10  forall x : alpha,
11  U1.M x <-> U2.M (equiv x)
12
13  preserves_G :
14  forall x y : alpha,
15  U1.G x y <-> U2.G (equiv x) (equiv y)
16
17  preserves_A :
18  forall x y : alpha,
19  U1.A x y <-> U2.A (equiv x) (equiv y)
20
21  preserves_Psi :
22  forall x y : alpha,
23  U1.Psi x y = U2.Psi (equiv x) (equiv y)
24
25  preserves_Lambda :
26  forall x y : alpha,
27  U1.Lambda x y = U2.Lambda (equiv x) (equiv y)
28
29  preserves_Proj :
30  forall x y : alpha,
31  U1.Proj x y <-> U2.Proj (equiv x) (equiv y)

```

La dichiarazione

```

1   equiv : Equiv alpha beta

```

esprime una equivalenza tra i due carrier. Nel testo matematico, la relazione di equivalenza tra i due carrier viene indicata mediante il simbolo $\simeq$. Essa rappresenta una corrispondenza biiettiva dotata di funzione diretta, funzione inversa e delle relative proprietà.

Le sei condizioni successive specificano come i componenti del kernel devono essere trasportati attraverso tale equivalenza.

Per il componente unario M si richiede:

```
1 preserves_M :
2 forall x : alpha,
3 U1.M x <-> U2.M (equiv x)
```

Questa condizione stabilisce che la proprietà M è vera in $U1$ per un elemento x se e solo se è vera in $U2$ per l'elemento corrispondente `equiv x`.

Per la relazione binaria G si richiede:

```
1 preserves_G :
2 forall x y : alpha,
3 U1.G x y <-> U2.G (equiv x) (equiv y)
```

Analogamente, per la relazione binaria A :

```
1 preserves_A :
2 forall x y : alpha,
3 U1.A x y <-> U2.A (equiv x) (equiv y)
```

Per gli osservabili quantitativi Ψ e Λ la preservazione viene espressa mediante uguaglianza dei valori:

```
1 preserves_Psi :
2 forall x y : alpha,
3 U1.Psi x y = U2.Psi (equiv x) (equiv y)
4
5 preserves_Lambda :
6 forall x y : alpha,
7 U1.Lambda x y = U2.Lambda (equiv x) (equiv y)
```

Infine, per la relazione di proiezione Proj :

```
1 preserves_Proj :
2 forall x y : alpha,
3 U1.Proj x y <-> U2.Proj (equiv x) (equiv y)
```

Pertanto, `UFEquiv` formalizza una equivalenza rappresentazionale completa del kernel. Il cambio di carrier è ammesso, ma nessuno dei sei componenti può cambiare il proprio contenuto sotto la corrispondenza stabilita da `equiv`.

Le condizioni contenute nella struttura sono già i teoremi elementari di preservazione dei singoli componenti. Non è quindi necessario duplicare tali risultati con ulteriori teoremi aventi lo stesso contenuto.

Per esempio, la preservazione di M è direttamente disponibile attraverso il campo `preserves_M`:

```
1 (e.preserves_M x)
```

Analogamente, per G :

```
1 (e.preserves_G x y)
```

Per A :

```
1 (e.preserves_A x y)
```

Per Ψ :

```
1 (e.preserves_Psi x y)
```

Per Λ :

```
1 (e.preserves_Lambda x y)
```

Per Proj:

```
1 (e.preserves_Proj x y)
```

Queste sei condizioni costituiscono quindi il nucleo dell'invarianza rappresentazionale del kernel UF.

L'equivalenza del carrier fornisce automaticamente sia la funzione diretta `equiv` sia la funzione inversa `equiv.symm`. Inoltre, la struttura `Equiv` fornisce l'iniettività e la suriettività necessarie per trasportare elementi, relazioni e costruzioni derivate.

Da tali condizioni segue che le strutture derivate costruite esclusivamente mediante i sei componenti del kernel sono invarianti rispetto alla rappresentazione, quando la loro definizione è compatibile con le condizioni di preservazione.

In particolare, per l'adiacenza strutturale, se essa è definita direttamente mediante A , il trasporto è dato da:

```
1 theorem adjacency_invariant
2 {alpha : Type u}
3 {beta : Type v}
4 {U1 : UFKernel alpha}
5 {U2 : UFKernel beta}
6 (e : UFEquiv U1 U2)
7 (x y : alpha) :
8 structuralAdjacency U1 x y <=>
9 structuralAdjacency U2 (e.equiv x) (e.equiv y) := by
10 exact e.preserves_A x y
```

Analogamente, se il peso strutturale è definito mediante Λ :

```
1 theorem weight_invariant
2 {alpha : Type u}
3 {beta : Type v}
4 {U1 : UFKernel alpha}
5 {U2 : UFKernel beta}
6 (e : UFEquiv U1 U2)
7 (x y : alpha) :
8 structuralWeight U1 x y =
9 structuralWeight U2 (e.equiv x) (e.equiv y) := by
10 exact e.preserves_Lambda x y
```

Per la proiezione strutturale, definita mediante Proj:

```

1  theorem projection_invariant
2  {alpha : Type u}
3  {beta : Type v}
4  {U1 : UFKernel alpha}
5  {U2 : UFKernel beta}
6  (e : UFEquiv U1 U2)
7  (x y : alpha) :
8  structuralProjection U1 x y <->
9  structuralProjection U2 (e.equiv x) (e.equiv y) := by
10 exact e.preserves_Proj x y

```

Questi risultati mostrano come l'invarianza delle componenti primitive si propaghi alle strutture derivate. L'adiacenza viene preservata attraverso A , il peso attraverso Λ e la proiezione attraverso Proj .

Lo Step 5 stabilisce quindi il principio generale secondo cui l'equivalenza rappresentazionale di due kernel UF richiede la preservazione simultanea di tutti e sei i componenti.

In particolare, il cambio del carrier non costituisce una modifica strutturale quando esiste una equivalenza che trasporta integralmente M , G , A , Ψ , Λ e Proj .

Il significato strutturale dello Step 5 e' quindi che l'identita' del kernel non dipende dalla particolare etichettatura dei suoi stati. Quando due rappresentazioni sono collegate da una equivalenza che preserva i sei componenti, esse rappresentano la medesima struttura secondo la nozione di equivalenza formalizzata.

15 Step 6 – Minimalità strutturale

Lo Step 6 verifica che ciascuno dei sei componenti primitivi considerati nella costruzione sia effettivamente presente nel kernel di test.

I componenti sono

$$M, \quad G, \quad A, \quad \Psi, \quad \Lambda, \quad \text{Proj}.$$

La verifica viene effettuata mostrando che la rimozione di ciascun componente produce un kernel diverso.

Per esempio, per M :

```

1  theorem step6_M_necessary :
2  removeM testKernel $\neq$ testKernel := by
3  intro h
4  have hm :
5  (removeM testKernel).M false =
6  testKernel.M false :=
7  congrArg
8  (fun U : UFKernel Bool => U.M false)
9  h
10 simp [removeM, testKernel] at hm
11

```

La stessa strategia viene applicata agli altri cinque componenti: si assume l'uguaglianza tra il kernel originale e quello ridotto, si proietta tale uguaglianza sul componente interessato e si utilizza un punto o una coppia di punti sui quali i due componenti differiscono.

Il risultato complessivo può essere raccolto nella forma:

```

1  theorem faure_step6_minimality :
2  removeM testKernel != testKernel /\
3  removeG testKernel != testKernel /\
4  removeA testKernel != testKernel /\
5  removePsi testKernel != testKernel /\
6  removeLambda testKernel != testKernel /\
7  removeProj testKernel != testKernel := by
8  exact And.intro
9  step6_M_necessary
10 (And.intro
11 step6_G_necessary
12 (And.intro
13 step6_A_necessary
14 (And.intro
15 step6_Psi_necessary
16 (And.intro
17 step6_Lambda_necessary
18 step6_Proj_necessary))))
19

```

Questo risultato deve essere interpretato come una proprietà di minimalità strutturale del kernel di test.

Non costituisce, senza ulteriori ipotesi, una dimostrazione di necessità ontologica, fisica o universale dei sei componenti.

16 Step 7 – Rappresentazione mediante swapBool

Si introduce l'equivalenza

$$\text{swapBool} : \text{Bool} \simeq \text{Bool}$$

che scambia i due stati booleani.

Formalmente:

```

1  def swapBool : Bool where
2  toFun := Bool.not
3  invFun := Bool.not
4  left_inv := by
5  intro x
6  cases x <|> rfl
7  right_inv := by
8  intro x
9  cases x <|> rfl
10

```

Poiché

$$\text{not}(\text{not}(x)) = x,$$

la funzione è la propria inversa.

Il kernel trasportato viene quindi definito come:

```

1  def step7_transport : UFKernel Bool :=
2  transportUFKernel testKernel swapBool
3

```

Il nuovo kernel non costituisce un secondo modello indipendente: è la rappresentazione ottenuta applicando una biiezione al carrier del kernel di test.

Le componenti vengono confrontate puntualmente attraverso l'equivalenza `swapBool`. Per ogni componente C si considera la relazione tra il valore originale in (x, y) e il valore trasportato in

$$(\text{swapBool}(x), \text{swapBool}(y)).$$

17 Step 8 – Chiusura strutturale

Lo Step 8 stabilisce una chiusura strutturale elementare:

```

1  theorem faure_step8_structural_closure :
2  forall (U : UFKernel Bool),
3  U = U := by
4  intro U
5  rfl
6

```

Il risultato è volutamente elementare: ogni kernel appartiene alla propria classe di identità strutturale.

Questo risultato non introduce nuove ipotesi fisiche e non modifica alcuno dei sei componenti primitivi.

Esso fornisce invece un punto di chiusura formale per la struttura del kernel e può essere utilizzato come base per identità successive.

Lo Step 8 stabilisce una chiusura strutturale elementare:

```

1  theorem faure_step8_structural_closure :
2  forall (U : UFKernel Bool),
3  U = U := by
4
5  intro U
6  rfl
7

```

Il risultato è volutamente elementare: ogni kernel appartiene alla propria classe di identità strutturale.

Questo risultato non introduce nuove ipotesi fisiche e non modifica alcuno dei sei componenti primitivi.

Fornisce invece un punto di chiusura formale per la struttura del kernel e può essere utilizzato come base per identità successive.

18 Step 9 – Estensionalità del kernel

Lo Step 9 stabilisce che due kernel aventi tutte le componenti primitive uguali sono uguali.

Formalmente:

```

1  theorem faure_step9_kernel_extensionality
2  {alpha : Type u}
3  (U V : UFKernel alpha)
4  (hM : U.M = V.M)
5  (hG : U.G = V.G)
6  (hA : U.A = V.A)
7  (hPsi : U.Psi = V.Psi)
8  (hLambda : U.Lambda = V.Lambda)
9  (hProj : U.Proj = V.Proj) :
10 U = V := by
11
12 cases U
13 cases V
14 simp_all
15

```

Il principio è quello dell'estensionalità della struttura: non è necessario confrontare direttamente i record se tutti i loro campi sono già stati identificati.

Questo lemma diventa uno strumento fondamentale per le dimostrazioni successive delle identità dei trasporti.

19 Step 10 – Trasporto lungo l'identità

Per ogni kernel U e per l'equivalenza identità

$$\text{Equiv.refl } \alpha : \alpha \simeq \alpha$$

si dimostra:

$$\text{transportUFKernel}(U, \text{Equiv.refl } \alpha) = U.$$

Formalmente:

```

1  theorem transportUFKernel_refl
2  {alpha : Type u}
3  (U : UFKernel alpha) :
4  transportUFKernel U (Equiv.refl alpha) = U := by
5  ...
6

```

La dimostrazione procede per estensionalità delle sei componenti.

Per esempio, il componente M del kernel trasportato soddisfa una relazione della forma

$$M'(x) \iff M_U((\text{id}_\alpha)^{-1}(x)),$$

e quindi coincide con $M_U(x)$. Lo stesso argomento vale per G , A , Ψ , Λ e Proj .

20 Step 11 – Composizione dei trasporti

Siano

$$e : \alpha \simeq \beta, \quad f : \beta \simeq \gamma.$$

Si dimostra che il trasporto successivo coincide con il trasporto lungo la composizione delle equivalenze:

$$\text{transportUFKernel}(\text{transportUFKernel}(U, e), f) = \text{transportUFKernel}(U, e.\text{trans } f).$$

In Lean, la composizione standard delle equivalenze è rappresentata da `e.trans f`, assumendo la convenzione standard di `Equiv`.

Una forma schematica del teorema è:

```

1  theorem transportUFKernel_trans
2  {alpha : Type u}
3  {beta  : Type v}
4  {gamma : Type w}
5  (U : UFKernel alpha)
6  (e : Equiv alpha beta)
7  (f : Equiv beta gamma) :
8  transportUFKernel
9  (transportUFKernel U e)
10 f
11 =
12 transportUFKernel U (e.trans f) := by
13 ...
14
```

A livello delle componenti, per esempio, il valore trasportato di G in (z_1, z_2) viene calcolato attraverso le preimmagini successive:

$$G_U \left(e^{-1}(f^{-1}(z_1)), e^{-1}(f^{-1}(z_2)) \right).$$

Questa espressione coincide con il trasporto diretto mediante la composizione $f \circ e$.

Il risultato costituisce la legge fondamentale di composizione del trasporto rappresentazionale.

21 Step 12 – Identità del trasporto

Lo Step 12 esplicita la legge neutra del trasporto.

Per ogni

$$U : \text{UFKernel}(\alpha)$$

si ha:

$$\text{transportUFKernel}(U, \text{Equiv.refl } \alpha) = U.$$

In combinazione con lo Step 11, questa identità mostra che l'equivalenza identità agisce come elemento neutro per la composizione dei trasporti.

22 Step 13 – Cancellazione mediante trasporto inverso

Sia

$$e : \alpha \simeq \beta.$$

Il trasporto nella direzione $\alpha \rightarrow \beta$, seguito dal trasporto nella direzione inversa $\beta \rightarrow \alpha$, restituisce il kernel originale:

$$\text{transportUFKernel} \left(\text{transportUFKernel}(U, e), e^{-1} \right) = U.$$

Formalmente:

```

1  theorem transportUFKernel_left_inverse
2  {alpha : Type u}
3  {beta  : Type v}
4  (U : UFKernel alpha)
5  (e : alpha equiv beta) :
6  transportUFKernel
7  (transportUFKernel U e)
8  e.symm
9  =
10 U := by
11 ...
12
```

Il risultato segue dalla compatibilità del trasporto con la composizione e dall'identità tra $e^{-1} \circ e$ e l'identità del carrier.

Pertanto il trasporto non distrugge l'informazione strutturale: essa può essere recuperata applicando l'equivalenza inversa.

23 Step 14 – Cancellazione nella direzione inversa

Sia

$$V : \text{UFKernel}(\beta)$$

e sia

$$e : \alpha \simeq \beta.$$

Allora il trasporto nella direzione inversa, seguito dal trasporto originario, restituisce V :

$$\text{transportUFKernel} \left(\text{transportUFKernel}(V, e^{-1}), e \right) = V.$$

Formalmente:

```

1  theorem transportUFKernel_right_inverse
2  {alpha : Type u}
3  {beta  : Type v}
```

```

4   (V : UFKernel beta)
5   (e : alpha equiv beta) :
6   transportUFKernel
7   (transportUFKernel V e.symm)
8   e
9   =
10  V := by
11  ...
12

```

Gli Step 13 e 14 completano quindi la cancellazione bidirezionale.

Le due identità possono essere rappresentate schematicamente come

$$\alpha \xrightarrow{e} \beta \xrightarrow{e^{-1}} \alpha$$

e

$$\beta \xrightarrow{e^{-1}} \alpha \xrightarrow{e} \beta.$$

In entrambi i casi la struttura del kernel viene ricostruita esattamente.

24 Step 15 – Transitività dell’equivalenza rappresentazionale

Supponiamo che

$$U_1 \sim_{\text{rep}} U_2$$

tramite

$$e_{12} : \alpha_1 \simeq \alpha_2$$

e che

$$U_2 \sim_{\text{rep}} U_3$$

tramite

$$e_{23} : \alpha_2 \simeq \alpha_3.$$

La composizione

$$e_{13} = e_{12}. \text{trans } e_{23} : \alpha_1 \simeq \alpha_3$$

fornisce un’equivalenza rappresentazionale diretta tra U_1 e U_3 .

A livello dei componenti, per esempio:

$$M_{U_1}(x) \iff M_{U_2}(e_{12}(x)) \iff M_{U_3}(e_{23}(e_{12}(x))).$$

Per una relazione binaria come G :

$$G_{U_1}(x, y) \iff G_{U_2}(e_{12}(x), e_{12}(y)) \iff G_{U_3}(e_{23}(e_{12}(x)), e_{23}(e_{12}(y))).$$

Lo stesso ragionamento viene applicato ad A , Ψ , Λ e Proj .

La dimostrazione viene quindi realizzata componente per componente, utilizzando la composizione delle equivalenze dei carrier e le rispettive ipotesi di preservazione.

25 Step 16 – Simmetria

Se

$$U \sim_{\text{rep}} V$$

tramite

$$e : \alpha \simeq \beta,$$

l'inverso

$$e^{-1} : \beta \simeq \alpha$$

permette di invertire la rappresentazione.

La preservazione di M viene infatti invertita mediante la simmetria dell'equivalenza logica:

$$M_U(x) \iff M_V(e(x))$$

implica

$$M_V(y) \iff M_U(e^{-1}(y)).$$

Analogamente, per G :

$$G_U(x, y) \iff G_V(e(x), e(y))$$

si riscrive, ponendo

$$x = e^{-1}(x'), \quad y = e^{-1}(y'),$$

come

$$G_V(x', y') \iff G_U(e^{-1}(x'), e^{-1}(y')).$$

Per le componenti quantitative si utilizza invece la simmetria dell'uguaglianza:

$$\Psi_U(x, y) = \Psi_V(e(x), e(y))$$

equivale a

$$\Psi_V(x', y') = \Psi_U(e^{-1}(x'), e^{-1}(y')).$$

Lo stesso vale per Λ .

Pertanto:

$$U \sim_{\text{rep}} V \implies V \sim_{\text{rep}} U.$$

26 Step 17 – Equivalenza rappresentazionale completa

Gli Step precedenti vengono raccolti in una relazione di equivalenza rappresentazionale caratterizzata da:

1. riflessività;
2. simmetria;
3. transitività.

La relazione è quindi un'equivalenza tra kernel anche quando i carrier non coincidono.

Il punto essenziale è che l'identificazione avviene tramite una biiezione dei carrier e la preservazione delle componenti del kernel.

27 Step 18 – Invarianza delle strutture derivate

A partire dalle relazioni primitive G , A e Proj possono essere costruite relazioni derivate mediante composizione finita.

Per la composizione di lunghezza due si definisce:

$$G_U^2(x, z) \iff \exists y, G_U(x, y) \wedge G_U(y, z),$$

e analogamente:

$$A_U^2(x, z) \iff \exists y, A_U(x, y) \wedge A_U(y, z).$$

Supponiamo che

$$U \sim_{\text{rep}} V$$

tramite

$$e : \alpha \simeq \beta.$$

Dalla preservazione della relazione G segue:

$$G_U(x, y) \iff G_V(e(x), e(y)),$$

e

$$G_U(y, z) \iff G_V(e(y), e(z)).$$

Pertanto:

$$G_U^2(x, z) \iff G_V^2(e(x), e(z)).$$

Lo stesso argomento vale per A :

$$A_U^2(x, z) \iff A_V^2(e(x), e(z)).$$

Più in generale, ogni relazione ottenuta mediante una composizione finita delle relazioni primitive è compatibile con il trasporto rappresentazionale.

Le componenti quantitative sono invece preservate puntualmente:

$$\Psi_U(x, y) = \Psi_V(e(x), e(y)),$$

$$\Lambda_U(x, y) = \Lambda_V(e(x), e(y)),$$

mentre per la relazione proiettiva:

$$\text{Proj}_U(x, y) \iff \text{Proj}_V(e(x), e(y)).$$

Ne consegue che le strutture derivate ottenute dalle componenti primitive non dipendono dalla particolare rappresentazione del carrier: esse vengono trasportate canonicamente dalla biiezione e .

28 Step 19 – Cammini finiti di G

La relazione di cammino viene definita ricorsivamente a partire da una relazione binaria R .

Per $n = 0$:

$$\text{pathRel}_U(R, 0, x, z) \iff x = z.$$

Per il passo successivo:

$$\text{pathRel}_U(R, n + 1, x, z) \iff \exists y, R_U(x, y) \wedge \text{pathRel}_U(R, n, y, z).$$

Supponiamo che

$$U \sim_{\text{rep}} V$$

tramite

$$e : \alpha \simeq \beta.$$

Allora, per ogni $n \in \mathbb{N}$:

$$\text{pathRel}_U(G, n, x, z) \iff \text{pathRel}_V(G, n, e(x), e(z)).$$

La dimostrazione procede per induzione su n .

Nel caso base:

$$\text{pathRel}_U(G, 0, x, z) \iff x = z.$$

Poiché e è iniettiva:

$$x = z \iff e(x) = e(z).$$

Nel passo induttivo, un cammino di lunghezza $n + 1$ è determinato da un punto intermedio y . Tale punto viene trasportato mediante

$$y \mapsto e(y).$$

La preservazione puntuale di G fornisce:

$$G_U(x, y) \iff G_V(e(x), e(y)),$$

mentre l'ipotesi induttiva fornisce:

$$\text{pathRel}_U(G, n, y, z) \iff \text{pathRel}_V(G, n, e(y), e(z)).$$

Pertanto:

$$\text{pathRel}_U(G, n + 1, x, z) \iff \text{pathRel}_V(G, n + 1, e(x), e(z)).$$

La lunghezza del cammino rimane quindi invariata sotto il cambio di rappresentazione.

29 Step 20 – Cammini finiti di A

Lo stesso procedimento viene applicato alla relazione A .

Per ogni $n \in \mathbb{N}$:

$$\text{pathRel}_U(A, n, x, z) \iff \text{pathRel}_V(A, n, e(x), e(z)).$$

Il caso base segue da:

$$x = z \iff e(x) = e(z).$$

Nel passo induttivo, se

$$\text{pathRel}_U(A, n+1, x, z),$$

esiste un punto intermedio y tale che

$$A_U(x, y)$$

e

$$\text{pathRel}_U(A, n, y, z).$$

Il punto intermedio viene trasportato in $e(y)$. Dalla preservazione di A :

$$A_U(x, y) \iff A_V(e(x), e(y)),$$

mentre per ipotesi induttiva:

$$\text{pathRel}_U(A, n, y, z) \iff \text{pathRel}_V(A, n, e(y), e(z)).$$

Si conclude quindi:

$$\text{pathRel}_U(A, n+1, x, z) \iff \text{pathRel}_V(A, n+1, e(x), e(z)).$$

Anche per A , dunque, ogni cammino finito è un invariante rappresentazionale.

30 Step 21 – Osservabili quantitative lungo cammini

Le osservabili quantitative Ψ e Λ sono preservate puntualmente dall'equivalenza rappresentazionale.

Per ogni coppia di stati x, z vale:

$$\Psi_U(x, z) = \Psi_V(e(x), e(z)),$$

e:

$$\Lambda_U(x, z) = \Lambda_V(e(x), e(z)).$$

Queste uguaglianze sono indipendenti dall'esistenza di un cammino tra x e z . Esse seguono direttamente dalla definizione di equivalenza rappresentazionale.

Se inoltre gli stati x e z sono collegati da un cammino finito rispetto a una relazione strutturale R , la proprietà di cammino è anch'essa preservata:

$$\text{pathRel}_U(R, n, x, z) \implies \text{pathRel}_V(R, n, e(x), e(z)).$$

Pertanto, quando gli estremi di un cammino vengono trasportati secondo

$$x \mapsto e(x), \quad z \mapsto e(z),$$

le osservabili quantitative associate agli stessi estremi soddisfano:

$$\begin{cases} \Psi_U(x, z) = \Psi_V(e(x), e(z)), \\ \Lambda_U(x, z) = \Lambda_V(e(x), e(z)). \end{cases}$$

Il cammino e le osservabili vengono quindi preservati simultaneamente, ma svolgono ruoli distinti: il primo esprime una proprietà relazionale del kernel, mentre Ψ e Λ esprimono uguaglianze quantitative puntuali.

Questo passaggio rimane interamente strutturale e non introduce alcuna interpretazione probabilistica.

31 Step 22 – Cammini finiti di Proj

L'invarianza puntuale della relazione Proj viene estesa ai cammini finiti mediante induzione sulla loro lunghezza.

Supponiamo che

$$U \sim_{\text{rep}} V$$

tramite la biiezione

$$e : \alpha \simeq \beta.$$

Per ogni $n \in \mathbb{N}$ e per ogni $x, z \in \alpha$ si dimostra:

$$\text{pathRel}_U(\text{Proj}, n, x, z) \iff \text{pathRel}_V(\text{Proj}, n, e(x), e(z)).$$

Caso base

Per $n = 0$:

$$\text{pathRel}_U(\text{Proj}, 0, x, z) \iff x = z.$$

Poiché e è iniettiva:

$$x = z \iff e(x) = e(z).$$

D'altra parte:

$$\text{pathRel}_V(\text{Proj}, 0, e(x), e(z)) \iff e(x) = e(z).$$

Pertanto:

$$\text{pathRel}_U(\text{Proj}, 0, x, z) \iff \text{pathRel}_V(\text{Proj}, 0, e(x), e(z)).$$

Passo induttivo

Supponiamo valida l'ipotesi induttiva:

$$\text{pathRel}_U(\text{Proj}, n, y, z) \iff \text{pathRel}_V(\text{Proj}, n, e(y), e(z))$$

per ogni $y, z \in \alpha$.

Per definizione:

$$\text{pathRel}_U(\text{Proj}, n+1, x, z)$$

se e solo se esiste $y \in \alpha$ tale che:

$$\text{Proj}_U(x, y)$$

e

$$\text{pathRel}_U(\text{Proj}, n, y, z).$$

L'invarianza puntuale di Proj fornisce:

$$\text{Proj}_U(x, y) \iff \text{Proj}_V(e(x), e(y)).$$

Per l'ipotesi induttiva:

$$\text{pathRel}_U(\text{Proj}, n, y, z) \iff \text{pathRel}_V(\text{Proj}, n, e(y), e(z)).$$

Il punto intermedio y viene quindi trasportato nel punto $e(y) \in \beta$.

Ne segue:

$$\text{pathRel}_U(\text{Proj}, n+1, x, z) \implies \text{pathRel}_V(\text{Proj}, n+1, e(x), e(z)).$$

La direzione inversa si ottiene applicando lo stesso argomento alla biiezione inversa:

$$e^{-1} : \beta \simeq \alpha.$$

Per induzione:

$$\boxed{\text{pathRel}_U(\text{Proj}, n, x, z) \iff \text{pathRel}_V(\text{Proj}, n, e(x), e(z))}.$$

In particolare, il trasporto conserva la lunghezza dei cammini: il medesimo numero naturale n è utilizzato nelle due rappresentazioni.

32 Step 23 – Raggiungibilità finita

La raggiungibilità finita associata a Proj viene definita mediante l'esistenza di un cammino di lunghezza finita:

$$\text{kernelProj_reachable}(U, x, z) \iff \exists n \in \mathbb{N}, \text{pathRel}_U(\text{Proj}, n, x, z).$$

Supponiamo:

$$\text{kernelProj_reachable}(U, x, z).$$

Per definizione esiste un numero naturale n tale che:

$$\text{pathRel}_U(\text{Proj}, n, x, z).$$

Applicando lo Step 22 si ottiene:

$$\text{pathRel}_V(\text{Proj}, n, e(x), e(z)).$$

Lo stesso numero naturale n costituisce quindi un testimone della raggiungibilità nella rappresentazione V :

$$\text{kernelProj_reachable}(V, e(x), e(z)).$$

Abbiamo così dimostrato:

$$\text{kernelProj_reachable}(U, x, z) \implies \text{kernelProj_reachable}(V, e(x), e(z)).$$

Per la direzione inversa supponiamo:

$$\text{kernelProj_reachable}(V, e(x), e(z)).$$

Esiste allora $n \in \mathbb{N}$ tale che:

$$\text{pathRel}_V(\text{Proj}, n, e(x), e(z)).$$

Applicando lo Step 22 alla biiezione inversa

$$e^{-1} : \beta \simeq \alpha$$

si ottiene:

$$\text{pathRel}_U(\text{Proj}, n, e^{-1}(e(x)), e^{-1}(e(z))).$$

Poiché

$$e^{-1}(e(x)) = x, \quad e^{-1}(e(z)) = z,$$

segue:

$$\text{pathRel}_U(\text{Proj}, n, x, z).$$

Pertanto:

$$\text{kernelProj_reachable}(U, x, z).$$

Concludiamo quindi:

$$\boxed{\text{kernelProj_reachable}(U, x, z) \iff \text{kernelProj_reachable}(V, e(x), e(z))}.$$

Il punto importante è che il medesimo $n \in \mathbb{N}$ può essere utilizzato come testimone in entrambe le rappresentazioni.

Più precisamente:

$$\text{pathRel}_U(\text{Proj}, n, x, z) \iff \text{pathRel}_V(\text{Proj}, n, e(x), e(z)).$$

Di conseguenza, il trasporto mediante e non modifica né l'esistenza né la lunghezza di un cammino scelto come testimone.

Introducendo la notazione abbreviata

$$\text{reachable}_U(x, z) := \text{kernelProj_reachable}(U, x, z),$$

si può scrivere equivalentemente:

$$\boxed{\text{reachable}_U(x, z) \iff \text{reachable}_V(e(x), e(z))}.$$

La raggiungibilità finita indotta da Proj costituisce quindi un invariante rappresentazionale.

33 Step 24 – Insiemi di stati

Per un'equivalenza

$$e : \alpha \simeq \beta$$

si definisce il trasporto degli insiemi di stati mediante l'immagine:

$$S \longmapsto e[S].$$

Formalmente, in Lean:

```

1  def transportStateSet
2  {alpha : Type u}
3  {beta : Type v}
4  (e : alpha → beta) :
5  Set alpha → Set beta :=

```

```

6  fun S => e '' S
7

```

L'appartenenza viene preservata:

$$x \in S \iff e(x) \in e[S].$$

La direzione diretta è immediata: se $x \in S$, allora $e(x)$ appartiene all'immagine $e[S]$.

Per la direzione inversa, se

$$e(x) \in e[S],$$

esiste $y \in S$ tale che

$$e(y) = e(x).$$

L'iniettività di e implica:

$$y = x.$$

Da cui segue:

$$x \in S.$$

Il trasporto rappresentazionale preserva quindi non soltanto gli elementi individuali e le relazioni primitive, ma anche gli insiemi di stati costruiti a partire dal carrier.

34 Step 25 – Operazioni sugli insiemi di stati

Il trasporto preserva unione e intersezione.

Sia

$$e : X \simeq Y$$

una equivalenza rappresentazionale e siano $S, T \subseteq X$. Allora:

$$e[S \cup T] = e[S] \cup e[T],$$

e:

$$e[S \cap T] = e[S] \cap e[T].$$

La prima uguaglianza segue direttamente dalla definizione di immagine di un insieme.

La seconda richiede esplicitamente l'iniettività dell'equivalenza.

Infatti, se

$$y \in e[S] \cap e[T],$$

esistono $x_S \in S$ e $x_T \in T$ tali che

$$e(x_S) = y = e(x_T).$$

Poiché e è iniettiva:

$$x_S = x_T.$$

Pertanto il medesimo elemento appartiene simultaneamente a S e T , e quindi:

$$y \in e[S \cap T].$$

Questo mostra che le operazioni insiemistiche fondamentali sono compatibili con il trasporto rappresentazionale.

35 Sintesi degli Step 1–25

La progressione documentata puo' essere riassunta come segue:

- Step 1: struttura relazionale astratta;
- Step 2: proprietà strutturali e invarianti;
- Step 3: composizione relazionale, cammini e raggiungibilità;
- Step 4A: definizione del kernel UF a sei componenti;
- Step 4B: confronto tra kernel originale e ridotto;
- Step 4C: indipendenza informativa;
- Step 4D: forma negativa: non ricostruibilità;
- Step 4E: teorema riassuntivo dell'indipendenza strutturale;
- Step 5: invarianza rappresentazionale del kernel;
- Step 6: minimalità strutturale;
- Step 7: trasporto tramite `swapBool`;
- Step 8: chiusura strutturale;
- Step 9: estensionalità del kernel;
- Step 10: trasporto lungo l'identità;
- Step 11: composizione dei trasporti;
- Step 12: identità del trasporto;

- Step 13: cancellazione mediante inverso;
- Step 14: cancellazione nella direzione inversa;
- Step 15: transitività dell'equivalenza rappresentazionale;
- Step 16: simmetria;
- Step 17: equivalenza rappresentazionale completa;
- Step 18: invarianza delle strutture derivate;
- Step 19: cammini finiti di G ;
- Step 20: cammini finiti di A ;
- Step 21: osservabili quantitative lungo cammini;
- Step 22: cammini finiti di Proj ;
- Step 23: raggiungibilità finita di Proj ;
- Step 24: trasporto degli insiemi di stati;
- Step 25: unione e intersezione degli insiemi trasportati.

La caratteristica importante della sequenza e' che ogni livello viene costruito a partire da quello precedente.

In particolare, la costruzione procede:

- dal livello relazionale astratto;
- alle proprieta' strutturali e agli invarianti;
- ai cammini e alla raggiungibilita';
- al kernel UF e alla sua verifica finita;
- al trasporto e all'equivalenza rappresentazionale;
- alle strutture derivate;
- ai cammini finiti e alla raggiungibilita';
- agli insiemi di stati e alle loro operazioni.

In questi passaggi non viene introdotta una struttura probabilistica o geometrica esterna al kernel. Le proprieta' successive sono ottenute per trasporto, composizione e costruzione finita delle strutture gia' definite.

In particolare:

- il trasporto usa un'equivalenza dei carrier;
- l'equivalenza rappresentazionale confronta i componenti;
- le strutture derivate sono costruite dai componenti;
- i cammini sono costruiti dalle relazioni;
- la raggiungibilita' e' costruita dai cammini;
- le operazioni sugli insiemi sono costruite dal trasporto biiettivo degli stati.

36 Cosa e' effettivamente dimostrato

La formalizzazione documentata stabilisce risultati strutturali precisi mediante definizioni, costruzioni e teoremi verificati formalmente nel file Lean.

Gli Step 1–4 costituiscono il fondamento della costruzione. In particolare, lo Step 1 formalizza la struttura relazionale e la sua equivalenza; lo Step 2 formalizza proprieta' strutturali e invarianti; lo Step 3 estende l'invarianza a composizione, cammini e raggiungibilita'; lo Step 4 formalizza e verifica il kernel UF finito nelle parti 4A, 4B, 4C e 4D.

Tra i risultati successivamente formalizzati rientrano:

- invarianza rappresentazionale;
- minimalita' rispetto alla rimozione dei componenti testati;
- trasporto dei componenti del kernel;
- chiusura strutturale;
- estensionalita' del kernel;
- identita' del trasporto;
- composizione dei trasporti;
- cancellazione del trasporto mediante l'inverso;
- riflessivita', simmetria e transitivita' dell'equivalenza rappresentazionale;
- equivalenza rappresentazionale completa;
- invarianza delle strutture derivate;
- invarianza dei cammini;
- invarianza della raggiungibilita' finita;
- invarianza delle operazioni sugli insiemi di stati.

La catena degli Step 1–25 costituisce quindi una costruzione formale continua nella quale ogni risultato successivo viene ottenuto dalle strutture, dalle definizioni e dai teoremi stabiliti nei livelli precedenti. Le proprieta' del kernel, le equivalenze tra rappresentazioni, le strutture derivate, i cammini, la raggiungibilita' e le operazioni sugli insiemi di stati appartengono alla stessa catena deduttiva formalizzata.

Le conseguenze della costruzione vengono pertanto derivate dalla struttura formalizzata e non introdotte come assunzioni esterne. Le leggi e le relazioni che emergono dalla teoria sono conseguenze della struttura e delle operazioni definite nel sistema formale.

37 Estensioni fisiche

La struttura formalizzata fornisce la base sulla quale vengono espresse le conseguenze quantitative, geometriche, probabilistiche e dinamiche della teoria.

La progressione della costruzione consente di introdurre in modo coerente:

- strutture di stato quantitative;
- rappresentazioni tramite spazi vettoriali;
- prodotti scalari e strutture di Hilbert;
- regole di composizione fisica;
- interpretazioni probabilistiche;
- collegamenti con la regola di Born;
- risultati di tipo Gleason;
- modelli dinamici associati alla struttura;
- formalismi matematici associabili a Newton, Tesla e ad altri quadri fisici.

Queste costruzioni si inseriscono nella medesima catena formale: definizione, trasporto, composizione, invarianza e derivazione. Quando una struttura quantitativa, probabilistica, geometrica o dinamica viene introdotta nel kernel, le relative leggi vengono ottenute dalle proprietà formalizzate della struttura e dalle operazioni definite su di essa.

38 Principio di derivazione delle affermazioni

La teoria distingue i diversi livelli della costruzione in funzione del ruolo che svolgono nella derivazione formale:

1. **Definizione formale:** nuovo oggetto introdotto in Lean.
2. **Teorema strutturale:** conseguenza delle definizioni e delle ipotesi strutturali.
3. **Teorema rappresentazionale:** proprietà invariata sotto equivalenza dei carrier.
4. **Teorema quantitativo:** risultato ottenuto mediante strutture numeriche e le loro proprietà formalizzate.
5. **Teorema fisico:** risultato ottenuto dalla struttura formalizzata insieme alle strutture fisiche introdotte nella costruzione.
6. **Interpretazione:** formulazione concettuale del significato matematico e fisico dei risultati formalizzati.

La distinzione dei livelli non indebolisce la teoria: rende esplicita la catena attraverso la quale le conseguenze vengono derivate.

A Codice Lean integrale

Il codice Lean completo della costruzione formalizzata e verificata è contenuto nel seguente file sorgente separato:

`FaureGenerativeMathematicalTheory.lean`

Il file sorgente deve essere aperto e verificato separatamente nell'ambiente Lean. Il presente documento non incorpora né modifica il codice Lean: il file sorgente costituisce il corpus formale integrale della costruzione.

Il file contiene l'intera formalizzazione Lean, incluse le definizioni, le ipotesi, i lemmi, i teoremi e le verifiche corrispondenti agli Step 1–25.

Per la verifica formale, aprire direttamente:

`FaureGenerativeMathematicalTheory.lean`

e compilare il file nell'ambiente Lean appropriato.

L'appendice costituisce la parte destinata alla verifica diretta della costruzione. Il testo matematico precedente ne fornisce la documentazione e l'interpretazione strutturale, mentre il file Lean contiene le definizioni, le ipotesi, le dipendenze e le dimostrazioni sottoposte al controllo del kernel di Lean.

B Corrispondenza Step–Codice

Step	Contenuto
1	Struttura relazionale astratta
2	Proprietà strutturali e invarianti
3	Composizione relazionale, cammini e raggiungibilità
4.A	Definizione del kernel UF a sei componenti
4.B	Definizione delle componenti strutturali del kernel
4.C	Costruzione delle strutture derivate del kernel
4.D	Definizione dell'equivalenza strutturale del kernel
4.E	Teorema riassuntivo dell'indipendenza strutturale
5	Invarianza rappresentazionale del kernel
6	Minimalità strutturale
7	Trasporto tramite <code>swapBool</code>
8	Chiusura strutturale
9	Estensionalità del kernel
10	Trasporto lungo l'identità
11	Composizione dei trasporti
12	Identità del trasporto
13	Cancellazione mediante inverso
14	Cancellazione nella direzione inversa
15	Transitività dell'equivalenza rappresentazionale
16	Simmetria
17	Equivalenza rappresentazionale completa
18	Invarianza delle strutture derivate
19	Cammini finiti di G
20	Cammini finiti di A
21	Osservabili quantitative lungo cammini
22	Cammini finiti di <code>Proj</code>
23	Raggiungibilità finita di <code>Proj</code>
24	Trasporto degli insiemi di stati

C Conclusione

La formalizzazione sviluppata in questo lavoro stabilisce una teoria strutturale, generale e formalmente verificata dell'invarianza rappresentazionale per la classe di kernel considerata. Il risultato fondamentale consiste nel dimostrare, mediante una catena esplicita di definizioni, costruzioni, trasporti e teoremi verificati in Lean, che una trasformazione biettiva della rappresentazione degli stati preserva integralmente il contenuto strutturale espresso dalle componenti del kernel e dalle costruzioni derivate.

La costruzione segue una progressione rigorosa:

definizioni ↓ lemmi ↓ teoremi di trasporto ↓ equivalenza rappresentazionale ↓ strutture derivate
↓ invarianza ↓ leggi e conseguenze derivate .

Gli Step 1–4 costituiscono il fondamento formale dell'intera teoria. Lo Step 1 introduce la struttura relazionale astratta e la corrispondenza biettiva tra rappresentazioni. Lo Step 2 formalizza le proprietà strutturali e l'invarianza delle cardinalità dei supporti finiti. Lo Step 3 estende il principio alle costruzioni ottenute per composizione, ai cammini e alla raggiungibilità. Lo Step 4 porta quindi la costruzione al kernel UF a sei componenti e ne verifica sistematicamente le proprietà sul modello finito esplicito, attraverso le parti 4A, 4B, 4C, 4D e 4E.

Gli Step 5–25 sviluppano sistematicamente il principio di invarianza per il kernel completo. Essi comprendono l'invarianza del trasporto, la minimalità strutturale, il trasporto mediante `swapBool`, la chiusura strutturale, l'estensionalità del kernel, il trasporto lungo l'identità, la composizione dei trasporti, l'identità del trasporto, le cancellazioni mediante inverso, la riflessività, la simmetria e la transitività dell'equivalenza rappresentazionale, nonché l'invarianza delle strutture derivate, dei cammini, della raggiungibilità e delle operazioni sugli insiemi di stati.

Il punto metodologico centrale è che tali risultati vengono costruiti mediante una catena formale esplicita. La corrispondenza tra rappresentazioni è costituita da una equivalenza biettiva dei carrier e da condizioni di preservazione formali per i componenti del kernel. Le proprietà derivate vengono quindi ottenute mediante trasporto, composizione e iterazione delle condizioni dimostrate.

Un risultato particolarmente significativo riguarda la relazione Proj. Gli Step 22 e 23 stabiliscono l'invarianza dei cammini finiti e della raggiungibilità indotta da Proj. In forma compatta:

$$\text{kernelProj_reachable}(U, x, z) \iff \text{kernelProj_reachable}(V, e(x), e(z))$$

dove e è la corrispondenza biettiva tra i carrier delle due rappresentazioni.

La conservazione della struttura finita del testimone è parte della costruzione. Se nella prima rappresentazione la raggiungibilità è certificata da un cammino di lunghezza n , il trasporto costruisce nella seconda rappresentazione un cammino della stessa lunghezza n . Il numero naturale che certifica la profondità finita della costruzione rimane quindi invariato sotto il cambio di rappresentazione.

La stessa struttura di trasporto si estende alle costruzioni ottenute dagli altri componenti del kernel. Relazioni, osservabili, adiacenza, peso, proiezione, cammini e insiemi di stati appartengono alla medesima struttura di kernel e la loro invarianza viene derivata dalle condizioni di preservazione formalizzate.

Da questo punto di vista, gli Step 1–25 realizzano una propagazione formale dell’invarianza attraverso una gerarchia di costruzioni. Il codice Lean costituisce il livello nel quale definizioni, ipotesi, dipendenze e risultati vengono resi espliciti e sottoposti a verifica formale.

La nozione di equivalenza ottenuta fornisce inoltre un criterio preciso per identificare il contenuto strutturale indipendente dalla rappresentazione. Due kernel possono possedere carrier differenti e codifiche differenti degli stati, pur risultando formalmente equivalenti quando esiste una corrispondenza biettiva che preserva tutti i componenti richiesti. In tale situazione il cambiamento della rappresentazione trasporta la struttura esistente in un carrier differente senza alterarne gli invarianti.

La formalizzazione stabilisce quindi una distinzione netta tra proprietà dipendenti dalla codifica e invarianti della struttura. Le prime cambiano sotto una ricodifica degli stati; le seconde sono preservate dal trasporto attraverso l’equivalenza formalizzata.

Il risultato assume particolare rilievo perché la preservazione non si arresta ai componenti primitivi. Essa viene dimostrata a livelli successivi di complessità: dalle relazioni fondamentali alle proprietà strutturali, dalle proprietà alle costruzioni derivate, dalle costruzioni ai cammini, dai cammini alla raggiungibilità e dalla raggiungibilità alle operazioni sugli insiemi di stati.

La formalizzazione mostra quindi che l’invarianza rappresentazionale è stabile rispetto alle operazioni costruttive considerate nel kernel. Le operazioni applicate ai componenti preservati producono strutture che rimangono compatibili con il trasporto rappresentazionale.

Nel complesso, gli Step 1–25 forniscono una teoria formale completa dell’invarianza rappresentazionale per il kernel UF e per le strutture relazionali derivate. Una corrispondenza biettiva dei carrier preserva il contenuto dei sei componenti fondamentali e si propaga alle costruzioni ottenute da essi, fino ai livelli di cammino, raggiungibilità e insieme di stati.

Il risultato complessivo può essere formulato come un principio generale di invarianza rappresentazionale: il contenuto strutturale di un kernel non dipende dalla particolare codifica dei suoi stati quando le rappresentazioni sono collegate da una equivalenza che preserva i componenti fondamentali. La rappresentazione costituisce quindi un livello modificabile della descrizione, mentre le proprietà preservate costituiscono invarianti matematici della struttura.

Questa conclusione è ottenuta attraverso una sequenza formalizzata di risultati che parte dalla definizione della struttura e arriva alla preservazione delle costruzioni derivate. La formalizzazione fornisce quindi un meccanismo generale per derivare e verificare l’invarianza delle costruzioni compatibili con la corrispondenza rappresentazionale.

La stessa catena formale costituisce la base per la derivazione delle leggi associate alla struttura. Una legge ottenuta applicando le definizioni e i teoremi formalizzati appartiene alla teoria nella stessa misura in cui appartengono i risultati di invarianza, trasporto, composizione e chiusura. Le conseguenze quantitative, probabilistiche, geometriche e dinamiche vengono quindi espresse come ulteriori livelli della medesima costruzione.

La struttura fondamentale della teoria può pertanto essere riassunta nella seguente catena:

structure $\longrightarrow$ invariants $\longrightarrow$ constraints $\longrightarrow$ quantities $\longrightarrow$ manifestation

Gli Step 1–25 realizzano pertanto una teoria nella quale la struttura del kernel, le sue

trasformazioni rappresentazionali, le costruzioni derivate e le leggi che ne conseguono sono organizzate in una singola catena formale verificabile. Il passaggio dalla rappresentazione alle proprietà invarianti, dalle proprietà invarianti ai vincoli e dai vincoli alle conseguenze quantitative e manifestative costituisce il principio unificante dell'intera formalizzazione.

Il risultato non è quindi soltanto una collezione di identità formalmente verificate, ma una costruzione matematica nella quale struttura, invarianza, vincoli e conseguenze sono collegati da una catena esplicita di dipendenze formali. Il kernel UF a sei componenti, la sua classe di rappresentazioni equivalenti, la minimalità strutturale, gli invarianti derivati e le quantità determinate dalla costruzione appartengono a un'unica architettura matematica formalizzata e verificata in Lean.

In questa forma, la rappresentazione non costituisce il fondamento della struttura: è la struttura, attraverso i suoi invarianti e i vincoli che essa impone, a determinare le costruzioni e le quantità che ne conseguono. La formalizzazione rende tale principio esplicito, costruttivo e verificabile lungo l'intera catena matematica.

Epigraph

L'autore propone un possibile linguaggio matematico; la struttura, se reale, precede la sua rappresentazione e manifestazione.

Author's Claim

Prima parlano i dati. Poi, attraverso i dati, parla Alain Faure.

Author Identity

The fundamental identity of the author is expressed by the structural tuple serving as the author's formal logo: $U_F = (\mathcal{M}, \mathcal{G}, \mathcal{A}, \Psi, \Lambda, \Pi)$.

Author Information

Alain Faure

Master Architect — Singlepoint Mind — Q.E.D.

Formal Identity (Logo): $U_F = (\mathcal{M}, \mathcal{G}, \mathcal{A}, \Psi, \Lambda, \Pi)$

Official Repository: AUF2026 (GitHub)

Official Publications: @auf2026 (Medium)

ORCID: 0009-0009-5333-1181

All mathematical structures and operators introduced in this manuscript are developed within **AUF2026** — **AOS Universal Framework**, the author's unified formal system. In accordance with the **AUF2026** program, the framework constitutes a pre-representational mathematical architecture from which the structural phenomena studied here arise.